MILESTONE 1 · ANALYTICAL REPORT

Formula 1 Podium Prediction

For every driver on the starting grid of a race, predict whether they finish in the top three, using only information known after qualifying and before the start.

Team 10	Omar Ahmed Fawzy Ahmed Amin · Khaled Mohamed Karam Alah Salleh · Youssef Ahmed Fathy Elsayed Elfeky · Muhamed Muhamed Muharam Mustafa

Kaggle notebook	**[add the public Kaggle notebook link]**
GitHub repository	github.com/MuhamedMuharam/ACL-Team-10
Dataset	Formula 1 World Championship (1950–2024), Kaggle / Ergast, 14 relational tables
Notebook	`F1_Podium_Prediction.ipynb`: one Run All, sections 1–13; every decision (D1–D30) is logged in `DECISIONS.md`

Summary

- **Data.** 26,759 raw result rows become **24,715 driver-race rows**: the cars on the grid at the prediction cutoff, one row per driver per race, with the Indianapolis 500 and failed qualifiers removed. Models are trained on 1994–2019, tuned on 2020–2021 and tested once on 2022–2024.
- **Q1, front-row conversion.** Front-row cars podiumed more often after 2014: **68.0%** → **75.9%** ($p = 0.013$). Suzuka, Yas Marina and Monaco convert best after 2014. Circuits are not statistically distinguishable from each other in either period, so a real change in the *ranking* cannot be shown. A front-row qualifier moved back by a penalty podiums only 28%, against 73% for one who starts there.
- **Q2, home advantage.** Comparing drivers who started from similar grid slots in the same era, home drivers podium **–0.8 percentage points** less often (95% CI –2.1 to +0.3), and the odds ratio is 0.93 (0.79–1.11). There is **no evidence of a home advantage**.
- **Q3, mechanical retirements.** They fell from **8.9% to 5.5% of race starts**. The highest rates in 2014–2021 were Caterham (21.2%), Lotus F1 (17.3%) and Toro Rosso (16.1%). In 2022–2024 they were Alfa Romeo (12.5%) and Alpine (10.4%), where team differences are only borderline.
- **Model.** The learned models reach validation PR-AUC 0.69–0.74, against 0.64 for a grid-only rule and 0.15 for a random model. None beats the **logistic-regression baseline** by more than noise, so the baseline is used for inference: **test PR-AUC 0.709**, ROC-AUC 0.936, well calibrated.
- **Explainability.** Starting position dominates, followed by season-long strength (standings). These explanations describe what the model relies on, **not what causes a podium**.

1. Data, audit and cleaning

1.1 Dataset, unit of analysis and the prediction cutoff

The data is spread over 14 linked tables. One row of the modeling table is **one driver entering one race**, and the target is `podium = positionOrder ≤ 3`. The prediction is made **after qualifying and before the start**. Anything produced by the race itself is leakage: finishing position, points, laps, status, pit stops, lap times, and the standings published after that race.

table	rows	use for race R	table	rows	use for race R
lap_times	589,081	post-race (not used)	races	1,125	pre-race (year, round, date, circuit)
driver_standings	34,863	previous race only	drivers	861	pre-race (date of birth, nationality)
results	26,759	grid only; the rest is the target or post-race	sprint_results	360	pre-race, but not used (D12)
constructor_standings	13,391	previous race only	constructors	212	lookup
constructor_results	12,625	not used	status	139	post-race (Q3 only)
pit_stops	11,371	post-race (not used)	circuits	77	pre-race (country)
qualifying	10,494	pre-race (position, Q1 time)	seasons	75	lookup

Table 1.1 The 14 tables and whether the current race's values may be used as features.

All files are first read **as plain text**, so the audit sees them exactly as written, with no type guessing and no silent NaN conversion. The raw files are never modified: an MD5 fingerprint of each file is taken at the start and checked after cleaning, and all 14 are byte-for-byte unchanged (D1). An assertion also checks the dataset version (last season 2024).

1.2 Audit of the raw data (before cleaning)

Sentinel values. Missing values are written as the literal text `\N`. Pandas reading the text sees **0 missing values**, so every one of them is hidden. Placeholders appear in 30 columns of 6 tables, and 68 cells are empty strings (all in `qualifying.q2/q3`), which mean the same thing. The missing values are *structural*:

- session dates exist only for recent seasons;
- finishing times are missing for non-finishers;
- `q2` / `q3` are missing for drivers knocked out earlier.

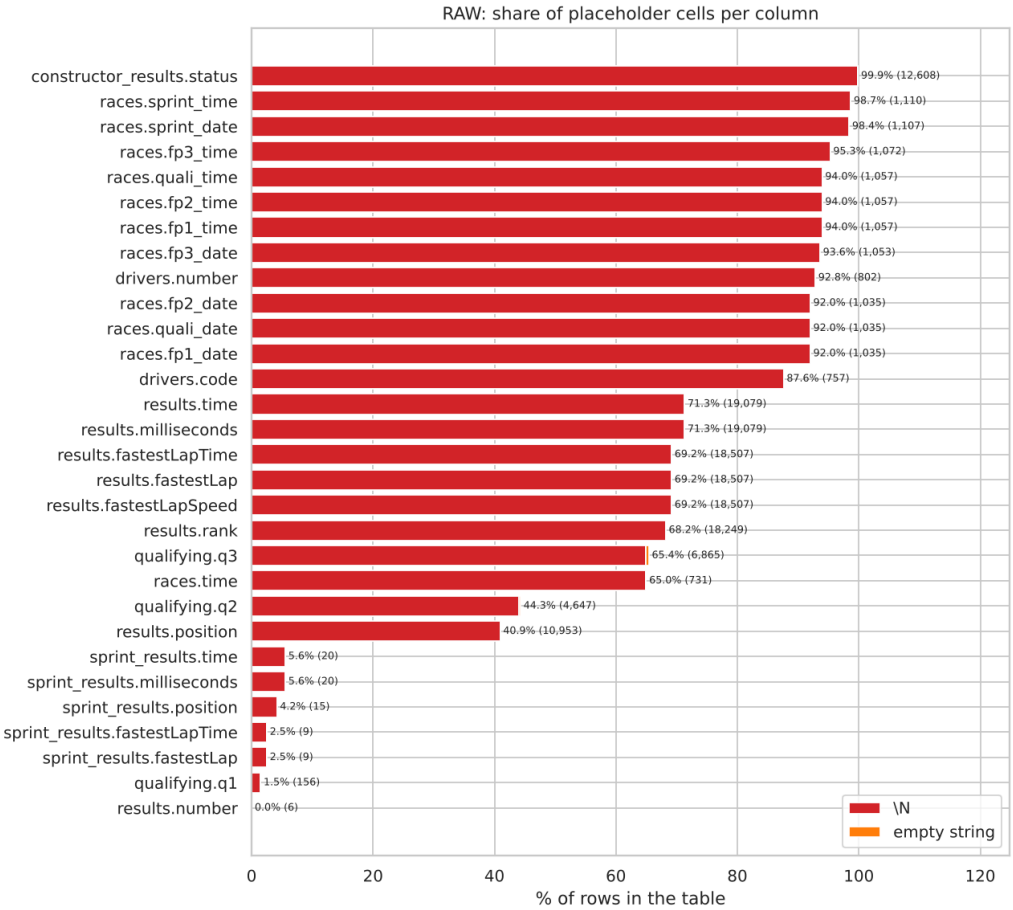


Figure 1.1 Raw data: share of placeholder cells (`\N` red, empty string orange) per column.

How the data changes over 75 seasons. Early races had far more entries than today's 20 cars: up to almost 40 rows per race around 1989, with failed qualifiers included. As a result, only about 8–12% of rows are podiums in the busy eras, against about 15% today. A raw `grid = 0` means two different things (Table 1.2): before 1995 it is almost always a driver who never got onto the grid, while from 1995 it is mostly a pit-lane start.

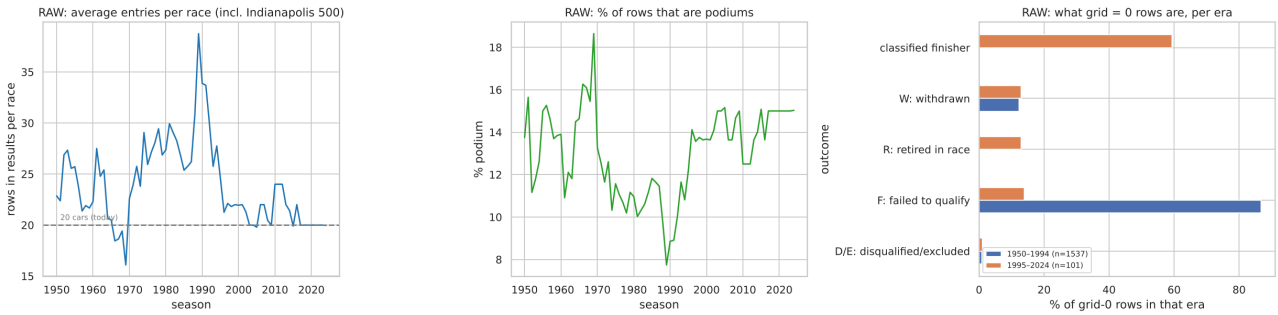


Figure 1.2 Raw data: (a) average entries per race, (b) share of rows that are podiums, (c) what `grid = 0` rows are, per era.

outcome of a raw grid-0 row	1950–1994	1995–2024
F: failed to qualify	1,334	14
W: withdrawn	190	13
D/E: disqualified or excluded	12	1
R: retired in the race	1	13
classified finisher	0	60

Table 1.2 The 1,638 raw rows with `grid = 0`. A grid 0 is a pit-lane start only for drivers who actually raced, which decides D6 and D7.

Qualifying coverage drift. `qualifying.csv` is empty before 1994. It covers 94% of races in 1994 and all of 1995, only 6–59% of races in 1996–2002, and every race from 2003. From 2003, at least 98.9% of result rows in every season have a qualifying row. So qualifying features are missing *by design* for most rows before 2003, and for those seasons the grid is the only record of the starting order.

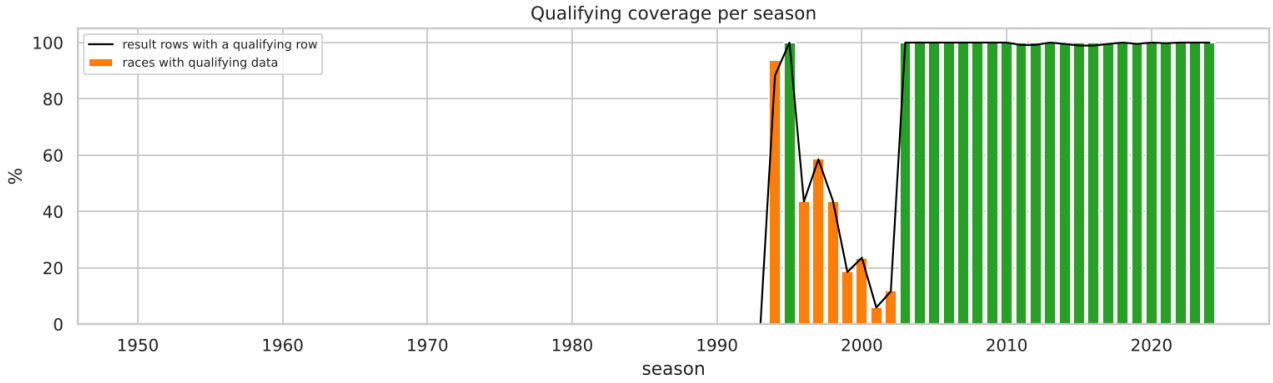


Figure 1.3 Share of races with qualifying data (bars; green = complete) and share of result rows with a qualifying row (line), per season.

Inconsistent text values were found by code checks, not by eye:

- the nationality `"Argentinian "` has a trailing space, and is the same nationality as `"Argentine"`;
- the circuit country appears both as `"United States"` and as `"USA"`;
- one constructor name contains an HTML code (`"Lotus-Pratt & Whitney"`).

Left alone, the first two would split one country into two groups in Q2. Dual or historical nationalities ("American-Italian", "East German") are real values, not errors. **Dates and durations stored as text** were also listed: dates; lap times such as `1:27.452`; the results `time` column, which mixes the winner's total time with gaps such as `+5.478`; pit-stop durations; and times of day.

1.3 Cleaning

step	what was done	result / check	decision
Sentinels → missing	Every file re-read with `\N` and `""` as missing; `keep_default_na=False` so real text such as "NA" is kept	Cell-by-cell check: NaN ⇔ placeholder; 0 cells disagree (asserted)	—
Semantic typing	Dates → datetime; lap, qualifying and fastest-lap times → seconds; pit durations → seconds; race times from `milliseconds`; `rank = 0` → missing	8 conversions, 0 parse failures (asserted); parsed lap times match the `milliseconds` column	—
Plausibility rule	A race time below laps × 30 s is impossible	18 rows (2024 Belgian GP, where the column holds gaps) set to missing	—
Text normalisation	Trailing spaces, "Argentinian" → "Argentine", "United States" → "USA", HTML codes unescaped	Nationalities 43 → 42, countries 35 → 34	D2
Primary and foreign keys	14 primary keys, 5 natural join keys, 26 foreign-key links (incl. lap / pit / qualifying rows → a result in that race)	All unique and non-null; **0 orphans** in all 26 links; every race has results	—
Status categories	Each of the 139 labels mapped explicitly to one of 8 categories (Table 1.4)	Asserted: every label mapped, none twice	D3
Grain: shared drives	85 (raceId, driverId) pairs (176 rows, 1950–1978); three keep rules compared (Table 1.5)	26,759 → 26,668 rows; (raceId, driverId) unique (asserted)	D4
Indianapolis 500	Removed (1950–1960): a separate competition	–381 rows	D5
Cars on the grid at the cutoff	Removed failed qualifiers and entries never on the grid; **kept** cars that qualified but did not start	–1,365 and –207 rows → **24,715 rows in 1,114 races**	D6
Starting position	Grid re-ranked 1 … field size among the kept cars; pit-lane starters to the last slot with a flag	73 pit-lane starts; 1 ≤ slot ≤ field size (asserted)	D7
Target	`podium = positionOrder ≤ 3`; `positionOrder` never used as a feature	Podium rate 13.5%; 1,113 races with 3 podium rows, 1 with 4	D8

Table 1.3 Cleaning steps in order, each with its count and the decision that justifies it (decisions in `DECISIONS.md`).

Status categories (D3). The brief defines mechanical as "a component of the car failed", which excludes crashes, the driver and regulation decisions. A label is `mechanical` only if it names a failed component. Damage usually caused by contact or the pit crew (tyres, wheels, wings, debris) is `ambiguous`, so those labels are treated the same way. "Withdrew" is decided per row in D6.

category	labels	result rows	examples
finished	35	15,333	Finished, +1 Lap, Not classified
mechanical	65	6,317	Engine, Gearbox, Hydraulics, Brakes, Power Unit
accident_driver	11	2,839	Accident, Collision, Spun off, Illness
did_not_qualify	3	1,365	Did not qualify, Did not prequalify, 107% Rule
ambiguous	18	489	Retired, Puncture, Wheel, Front wing, Debris
withdrawn	1	248	Withdrew
regulation	3	156	Disqualified, Excluded, Underweight
team_operational	3	12	Wheel nut, Fuel rig, Refuelling

Table 1.4 The 139 status labels in 8 categories, with how often each occurs in `results`.

Duplicate drivers in one race (D4). In the 1950s a driver could take over a team-mate's car mid-race, and then has two rows in the same race. At the 1957 British GP, Moss started car #18 from pole and later won in car #20, while Brooks drove #18 until its engine failed, so *both* drivers appear twice. Three keep rules were compared on two checks a clean table must pass:

rule	races with > 3 podium rows	drivers sharing one car
A: keep the best `positionOrder`	15	121
B: keep the driver's own entry (lowest `resultId`)	2	36
C: keep the car only he is recorded in, else B (chosen)	**1**	**30**

Table 1.5 Duplicate-rule comparison (lower is cleaner). All rules remove the same 67 non-Indy rows; they differ in which row survives.

Rule A credits a shared car's result to both drivers (Moss *and* Brooks "won" in 1957). It is also outcome-based row selection, which is a form of target leakage. Rule C keeps the car the driver **started** in, which is the car his pre-race information describes. For example, Ascari in 1951 started 3rd and retired, instead of "started 6th, finished 2nd" in his team-mate's car.

All affected rows are from before 1965, outside the training window, so the rule matters for the historical questions, not for the model.

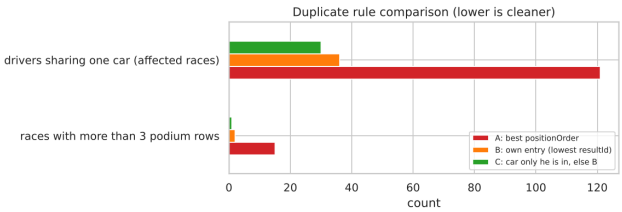


Figure 1.4 The three duplicate rules on the two integrity checks.

The Indianapolis 500 (D5). It counted for the championship, but of its 107 drivers only 4 ever started another championship race. Its entry lists average 35 per race (55 in 1954), against 22 at other races of that decade. Of its 381 rows, 380 are American drivers racing in the USA, more than three times all other American home starts in 75 seasons. Keeping it would distort field sizes, podium rates and the home-race question.

Who is in the table (D6). The prediction is made after qualifying. At that moment, failed qualifiers (1,365 rows) and entries that never reached the grid (207 rows) are already out. Cars that qualified but then failed to start are still on the grid and are **kept** as non-podiums (137 rows, e.g. Leclerc on pole at Monaco 2021): dropping them would use information from after the cutoff.

Starting position (D7). Left as 0, a pit-lane start would look better than pole. In old seasons the raw grid numbers also have holes and ties (1981 German GP: no slots 19, 20, 24, 25, 27, and the last of 24 starters on grid 28). The grid is therefore re-ranked among the cars on the grid, and pit-lane starters go to the last slot with their own flag.

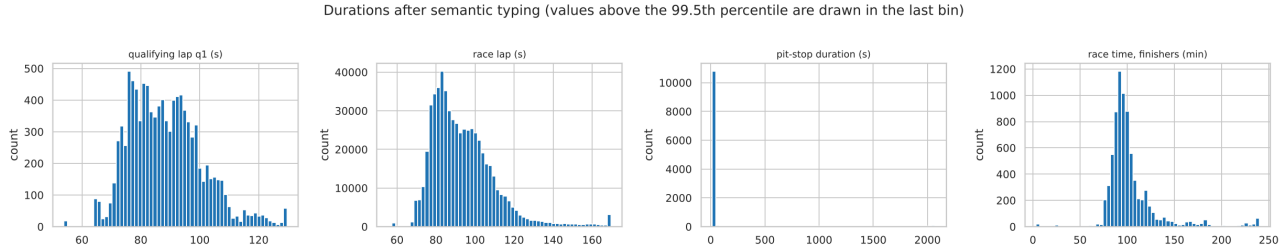


Figure 1.5 Durations after semantic typing. The long tails are real events: 708 laps over 5 minutes (red flags, safety cars) and 517 "pit stops" over a minute (red-flag stoppages). They are not features, so they are kept as they are. A few red-flag stoppages of up to about 50 minutes squeeze the normal 20–30 s pit stops into one bar.

1.4 EDA after cleaning: what the cleaning changed

	rows	duplicate (race, driver) rows	grid = 0 rows	slot > number of cars	podium rate	races
raw	26,759	91	1,638	13	12.69%	1,125
cleaned	24,715	0	0	0	13.53%	1,114

Table 1.6 The modeling grain before and after cleaning.

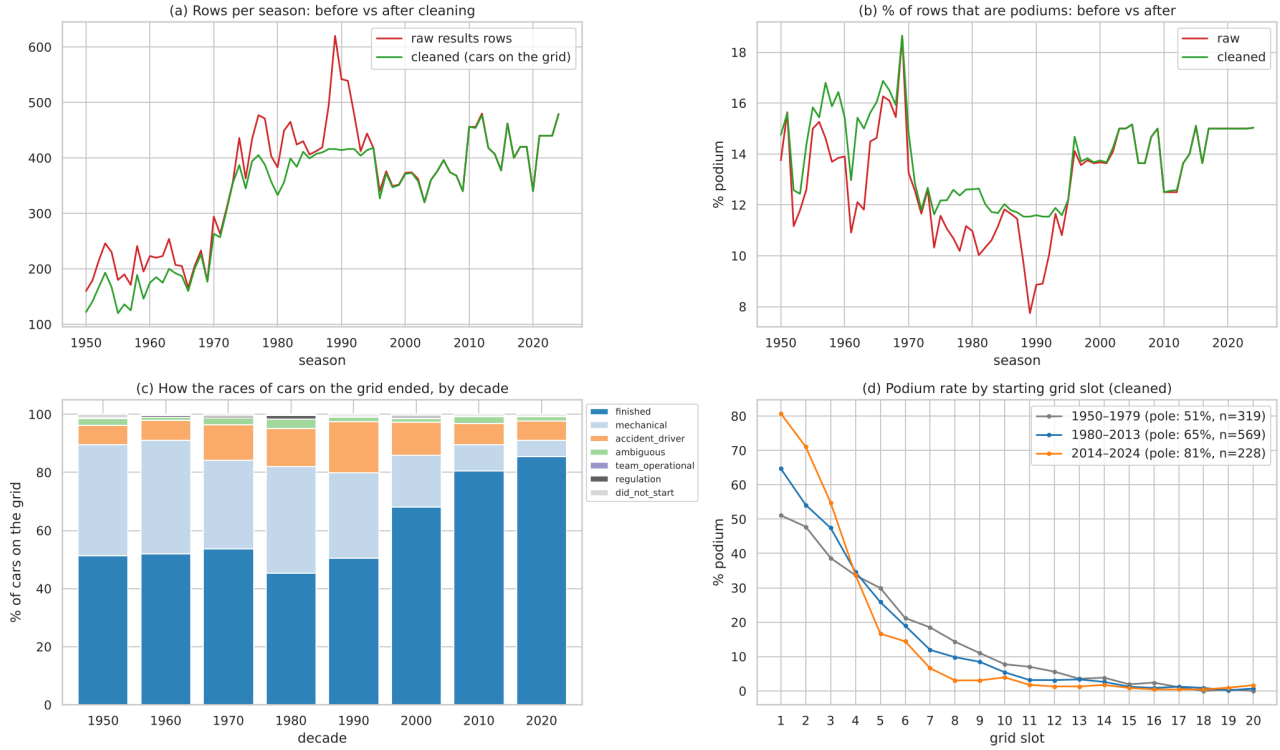


Figure 1.6 (a) Rows per season and (b) podium base rate, before vs after cleaning. (c) How the races of cars on the grid ended, by decade. (d) Podium rate by starting slot in three eras.

- **(a, b)** Cleaning removes rows mostly from the 1950s and the busy late-1970s to 1990s, where entry lists were far larger than the grid. Removing failed qualifiers, who can never podium, raises the base rate in those eras (late 1980s: from about 8–10% to about 12%). Modern seasons are almost untouched (about 15%).
- **(c)** Reliability changed enormously. In the 1950s–1980s around a third of the cars on the grid retired with a mechanical failure; in the 2020s most cars finish.
- **(d)** Starting position predicts podiums in every era, and much more strongly today. Pole converts 51% in 1950–1979, 65% in 1980–2013 and 81% in 2014–2024. This motivates the grid features and the training window.

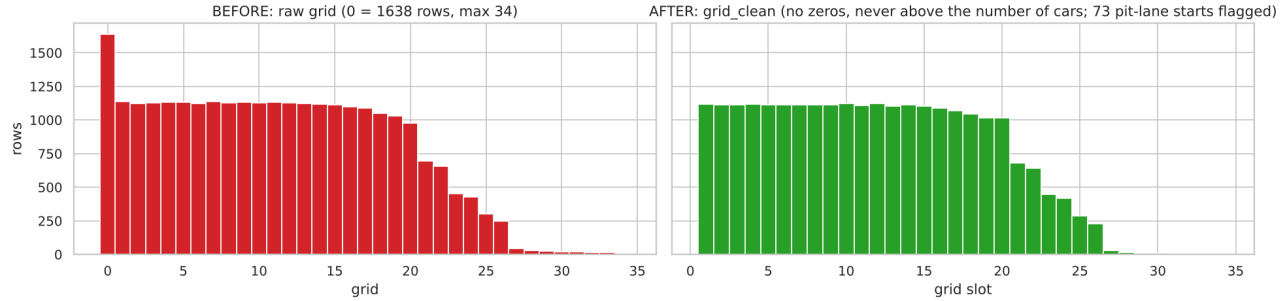


Figure 1.7 Starting position before (raw `grid`, with a spike of 1,638 zeros) and after the re-ranking (no zeros, never behind the last car).

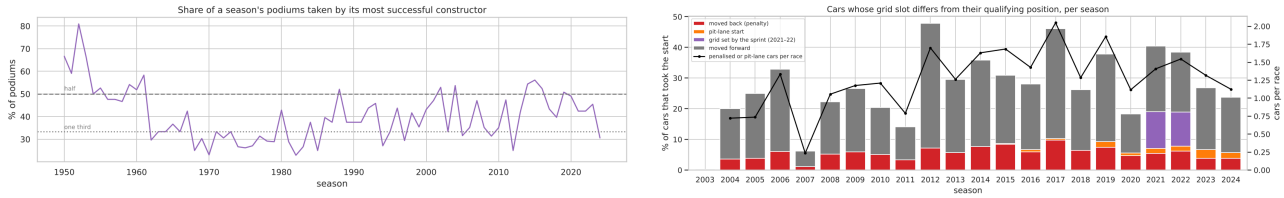


Figure 1.8 Left: the share of a season's podiums taken by its most successful constructor. The median is 37.5%, and above half in 15 seasons (e.g. 2014–2016, 2019). The car matters, so constructor features are included. Right: cars whose grid slot differs from their qualifying position. Penalised or pit-lane cars per race rose from 1.0 in 2003–2013 to 1.5 in 2014–2024. The earlier figure is about 1.1 without 2003, a season whose data shows no grid changes at all. On 2021–22 sprint weekends the sprint set the grid. Grid and qualifying are therefore both kept, with their difference as a feature.

2. Data-engineering questions

Each answer joins several tables, shows a plot, and states its grain, how missing values were handled, and whether the pre-race cutoff matters. All three use the cleaned driver-race table of Section 1 (starters only, no Indianapolis 500, one row per driver per race).

2.1 Q1. Which circuits convert a front-row start into a podium most often, and does the ranking change after 2014?

Joins: `results` ⋈ `races` ⋈ `circuits` ⋈ `qualifying` ⋈ `sprint_results`. **Set-up (D9):**

- The front row is measured two ways: by **grid** (slot 1–2 after penalties) and by **qualifying** (1st–2nd).
- The periods are 2003–2013 vs 2014–2024. 2003 is the first season with complete qualifying data, so both definitions use the same rows.
- A circuit is ranked only with at least 8 front-row starts (4 races) in both periods.
- Every rate gets a 95% Wilson interval. A chi-square test asks whether circuits differ at all, and the rank correlation gets a bootstrap interval.

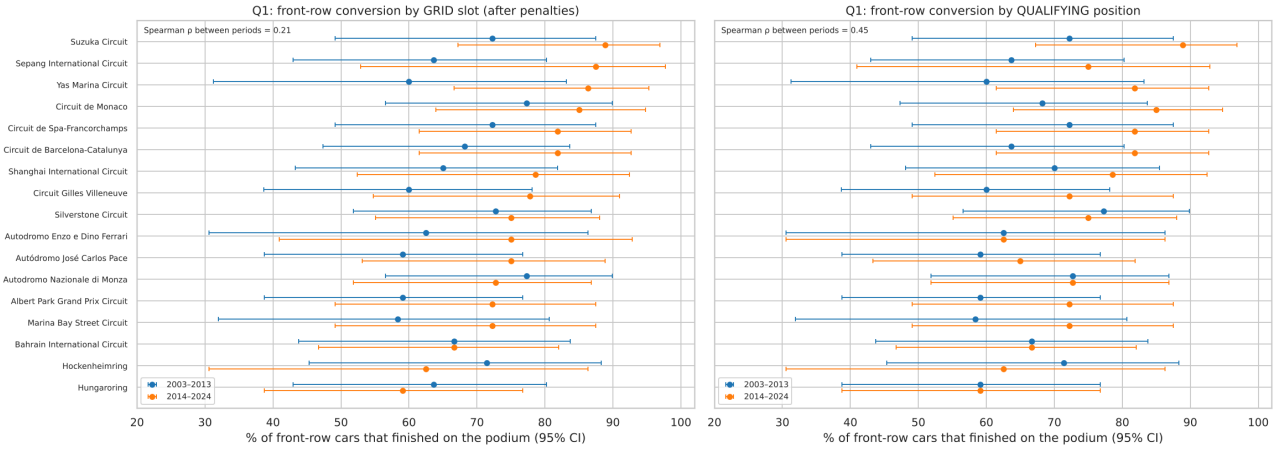
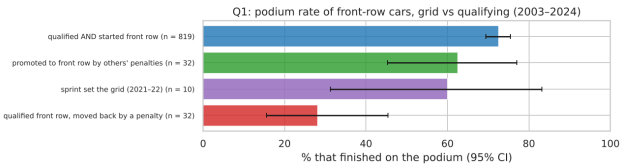


Figure 2.1 Front-row conversion per circuit with 95% intervals, 2003–2013 (blue) vs 2014–2024 (orange). Left: by grid slot. Right: by qualifying position (same circuit order).

circuit (grid definition)	2003–13 rate	rank	n	2014–24 rate	rank	n
Suzuka	72%	4	18	89%	1	18
Sepang	64%	10	22	88%	2	8
Yas Marina	60%	13	10	86%	3	22
Monaco	77%	1	22	85%	4	20
Spa-Francorchamps	72%	4	18	82%	5	22
Silverstone	73%	3	22	75%	9	24
Monza	77%	1	22	73%	12	22
Hungaroring	64%	10	22	59%	17	22

Table 2.1 Selected circuits from the full 17-circuit ranking (notebook §5.1): conversion rate, rank and number of front-row starts per period.



front-row case, 2003–2024	n	podium
qualified and started on the front row	819	72.5%
promoted by others' penalties	32	62.5%
sprint set the grid (2021–22)	10	60.0%
qualified front row, moved back by a penalty	32	28.1%

Table 2.2 Grid vs qualifying: why both definitions matter.

Figure 2.2 Podium rate of front-row cars by how their grid slot relates to their qualifying position (95% intervals).

Answer.

- **Front-row starts convert more often after 2014:** 68.0% (n = 400) → 75.9% (n = 456), chi-square p = 0.013.
- **Best converters.** After 2014: Suzuka (89%), Sepang (88%, only 8 starts), Yas Marina (86%) and Monaco (85%). Before: Monaco and Monza (77%) and Silverstone (73%).
- **The ranking appears to change, but the change cannot be shown.** Monza falls from joint 1st to 12th and Yas Marina rises from 13th to 3rd. But each circuit has only 8–24 starts per period, the circuits are not statistically distinguishable in either period (p = 0.99 and 0.80), and the rank correlation between periods is weak (ρ = 0.21, bootstrap CI −0.40 to 0.50). Monza's fall is *one podium fewer* (17 → 16 of 22), because all circuits sit between 58% and 89%. By qualifying position ρ = 0.45 (p = 0.07). The robust finding is the overall rise.
- **Grid vs qualifying matters.** A front-row qualifier moved back by a penalty podiums only 28%, against 73% for one who starts there (Table 2.2). The grid after penalties is therefore the better measure of the real starting chance.

Grain / missingness / cutoff. One row per driver-race, aggregated per circuit and period. The 22 rows from 2003–2024 without a qualifying position all started 19th or lower, so none is a front-row case; they are kept, not imputed. Grid, qualifying and the sprint result are all known before the race.

2.2 Q2. Do drivers podium more often at home, after controlling for starting position?

Joins: `results` ⋈ `races` ⋈ `circuits` (country) ⋈ `drivers` (nationality). **Set-up (D10):**

- A home race means the circuit's country matches the driver's nationality. The match uses an explicit hand-written map (e.g. "British" → "UK"), and both lists of values were checked for gaps.
- The circuit's location decides, not the race name, and a dual nationality counts both countries.
- Home and away are compared **within the same grid bucket** (1 / 2–3 / 4–6 / 7–10 / 11+) **× era** (1950–79 / 1980–2013 / 2014–24), weighted by home starts.
- Intervals come from 1,000 resamples of whole races. A logistic regression `podium ~ home + grid slot + era` with race-clustered standard errors is the robustness check.

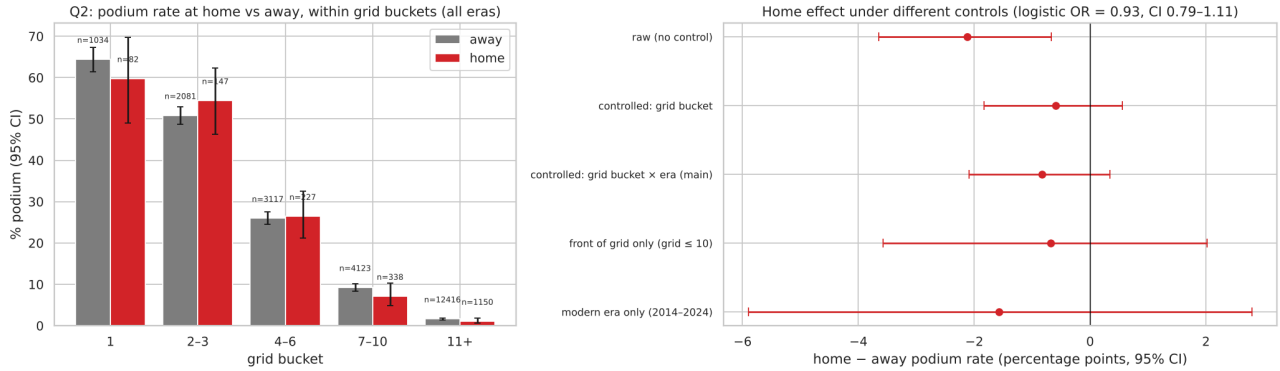


Figure 2.3 Left: podium rate at home (red) vs away (grey) within grid buckets, all eras. Right: the home – away difference under different controls, with 95% intervals.

estimate	home – away (pp)	95% CI	home starts
raw, no control	–2.12	–3.65 to –0.66	1,944
controlled: grid bucket	–0.60	–1.83 to +0.55	1,944
controlled: grid bucket × era (main)	**–0.83**	**–2.09 to +0.34**	**1,944**
front of the grid only (grid ≤ 10)	–0.68	–3.57 to +2.02	794
modern era only (2014–2024)	–1.58	–5.90 to +2.79	145

Table 2.3 Home effect in percentage points. Logistic regression (clustered by race): odds ratio 0.93 (95% CI 0.79–1.11), p = 0.43.

Answer.

- **No evidence that drivers podium more often at home once starting position is accounted for.**
- **The raw gap comes from who races at home.** Raw, home drivers podium 2.1 pp *less* often. 13.1% of home starts are by drivers with five or fewer career starts (one-off local entrants), against 1.4% of away starts.
- **Controlling for grid and era** shrinks the gap to –0.8 pp (CI –2.1 to +0.3). The logistic odds ratio is 0.93 (0.79–1.11), and the front of the grid alone gives –0.7 pp (CI –3.6 to +2.0).
- **Today's F1 is uncertain.** For 2014–2024 alone there are only 145 home starts, so the data cannot rule out an effect of a few points in either direction.
- This is an association, not a causal effect.

Grain / missingness / cutoff. One row per driver-race. Nationality and circuit country have no missing values; nationalities with no circuit in their country (e.g. Finnish) are always "away". Nationality, circuit and grid are known before the race, so `home_race` is a legal feature (context group).

2.3 Q3. Which constructors have the highest mechanical-retirement rate in 2014–2021 vs 2022–2024?

Joins: `results` ⋈ `races` ⋈ `status` (category, D3) ⋈ `constructors`. **Set-up (D11):**

- Rate = mechanical retirements ÷ **race starts**. Cars that were on the grid but did not take the start are not starts.
- The *strict* rate counts only labels that name a failed component. The *broad* rate adds ambiguous labels, as a robustness check, and a third version counts only failures after at least one lap.
- Teams are grouped by `constructorId`, so a renamed team is a separate entry.

- Only teams with ≥ 30 starts are ranked; rates have 95% Wilson intervals, and a chi-square test asks whether teams differ at all.

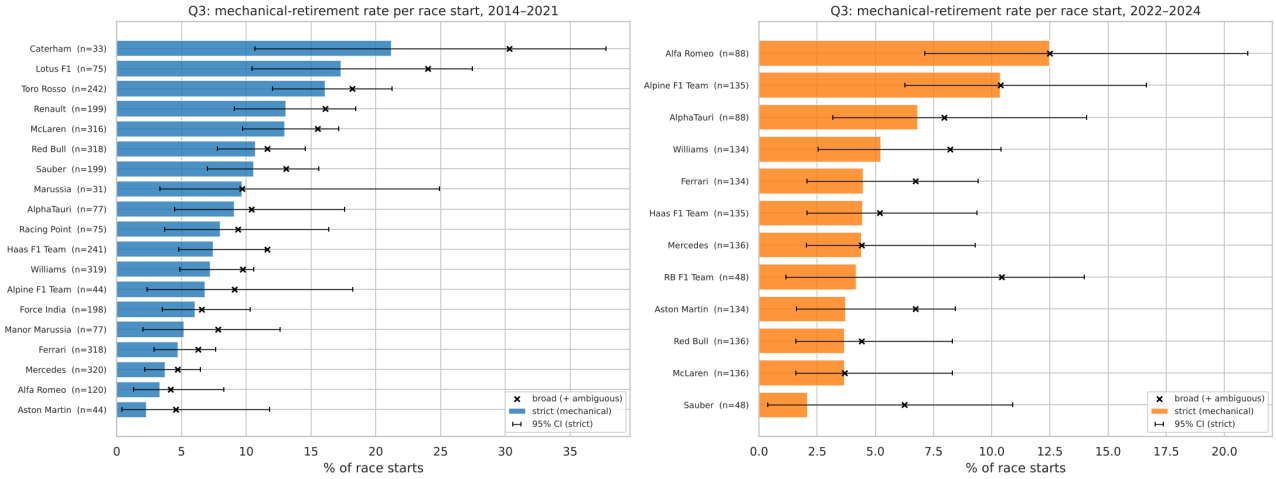


Figure 2.4 Mechanical-retirement rate per race start: bars = strict rate with 95% interval, × = broad rate (+ ambiguous labels), n = starts.

2014–2021 (overall 8.9%)	starts	strict	95% CI
Caterham	33	21.2%	10.7–37.8
Lotus F1	75	17.3%	10.4–27.4
Toro Rosso	242	16.1%	12.0–21.3
Renault	199	13.1%	9.1–18.5
McLaren	316	13.0%	9.7–17.1
…			
Mercedes	320	3.8%	2.2–6.4
Alfa Romeo	120	3.3%	1.3–8.3
Aston Martin	44	2.3%	0.4–11.8

2022–2024 (overall 5.5%)	starts	strict	95% CI
Alfa Romeo	88	12.5%	7.1–21.0
Alpine F1 Team	135	10.4%	6.3–16.7
AlphaTauri	88	6.8%	3.2–14.1
Williams	134	5.2%	2.6–10.4
…			
McLaren	136	3.7%	1.6–8.3
Red Bull	136	3.7%	1.6–8.3
Sauber	48	2.1%	0.4–10.9

Table 2.4 Highest and lowest strict rates per period (full tables with broad and after-lap-1 rates: notebook §5.3).

Answer.

- **2014–2021.** Caterham (21.2%), Lotus F1 (17.3%) and Toro Rosso (16.1%) had the highest mechanical-retirement rates, then Renault and McLaren (about 13%). Mercedes, the dominant team, was among the lowest (3.8%). Teams clearly differ ($p < 0.001$).
- **2022–2024.** Alfa Romeo (12.5%) and Alpine (10.4%) stand out. Below them the differences are only borderline ($p = 0.052$) and most intervals overlap.
- **Change between the periods.** Failures became rarer overall (8.9% → 5.5%), but not for everyone. McLaren (13.0% → 3.7%) and Red Bull (10.7% → 3.7%) improved sharply, while Alfa Romeo got worse (3.3% → 12.5%). For the 11 constructor entries present in both periods the rank correlation is −0.49 ($p = 0.13$), so there is no sign of a stable "unreliable team" across eras. This comparison follows `constructorId`, so renamed teams (Renault → Alpine, Toro Rosso → AlphaTauri) are not linked.
- **Robustness.** The broad rate keeps the 2014–2021 top five and Alfa Romeo first in 2022–2024. Counting only failures after at least one lap keeps the top two of both periods.

Grain / missingness / cutoff. One row per car that took the start, aggregated per constructor and period; status has no missing values. Status is **post-race** information: it is used only in this analysis, never as a feature for the current race. A team's *past* reliability is a legal feature when computed from earlier races only (`con_mech_rate_last10`). Renamed teams (Toro Rosso → AlphaTauri → RB, Force India → Racing Point → Aston Martin) appear as separate entries.

3. Features, split and preprocessing

3.1 Selected features, and why each is valid at the pre-race cutoff

The model uses **21 features in five groups**; the groups are reused by the ablation (Section 4.6). Each one is known after qualifying and before the start.

group	feature	definition	why it is known before the start
grid	`grid_pos`	starting slot after penalties, re-ranked among the cars on the grid	the starting grid is fixed after qualifying and penalties
	`grid_pct`	(slot − 1) ÷ (field size − 1): 0 = pole, 1 = last	
	`pit_lane_start`	flag: starts from the pit lane	
qualifying	`quali_position`	qualifying classification	qualifying runs before the race; on 2021–22 sprint weekends the sprint, also before the race, sets the reference
	`quali_gap_pct`	the driver's Q1 lap vs the fastest Q1 lap of that race, in % (laps > 20% off are not real attempts and become missing: 10 of 10,338)	
	`grid_penalty`	grid slot − qualifying position (on 2021–22 sprint weekends: − sprint result)	
standings	`drv_pts_share_before`, `con_pts_share_before`	championship points ÷ the leader's points	taken from the table published after the **previous** race of the season (season opener: the previous season's final table), never from the current race
	`drv_rank_before`, `con_rank_before`	championship position	
	`drv_win_share_before`, `con_win_share_before`	wins ÷ races run so far in the season	
	Shares and ranks are used instead of raw points because points systems (8, 9, 10, 25 for a win) and season lengths (7–24 races) changed.		
	A driver or team missing from the previous table gets 0 points, no wins and the rank after the last entry.		
	Championship exclusions decided at season end (Schumacher 1997) apply only to the final table.		
form	`drv_podium_rate_last5`	share of the driver's previous 5 races on the podium	rolling windows over **earlier** races only: `shift(1)` before `rolling()`, in date order; the team is collapsed to one row per race first, so a team-mate's current result never leaks in
	`drv_avg_finish_last5`	average finishing position as a fraction of the field, previous 5 races	
	`con_podium_rate_last5`	same as the driver version, for the team (both cars)	
	`con_avg_finish_last5`	same as the driver version, for the team (both cars)	
	`con_mech_rate_last10`	share of the team's cars with a mechanical failure, previous 10 races	

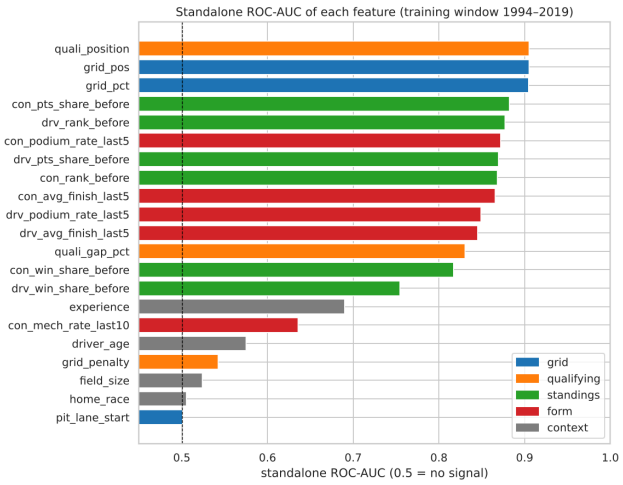
context	`driver_age`	age at the race date	date of birth, number of earlier races, nationality vs circuit country and the number of cars on the grid are all known beforehand
	`experience`	number of previous races on the grid	
	`home_race`	nationality matches the circuit's country (the Q2 map)	
	`field_size`	number of cars on the grid	

Table 3.1 The 21 features (D13–D16). Raw lap times are not used, because they partly measure track length (Monaco ≈ 70 s, Spa ≈ 105 s). Same-weekend sprint results are not used, because sprints are absent from every training season (D12). A "reached Q3" flag is not used: since 2006 it is essentially `quali_position ≤ 10`, and it does not exist before 2006.

3.2 Leakage controls

- **Standings from before the race.** At the 2023 Belgian GP, Verstappen's standings row says 314 points, which already includes that race. The feature uses the 281 he had before it. An assertion checks that every `prev_raceId` lies strictly earlier in time than its race.
- **Form from past races only.** `shift(1)` keeps the current race out of its own window, and races are ordered by date, not by `raceId`. A hand check recomputes Verstappen's podium rate over the 5 races before race 989 from the raw rows (0.40 = feature, asserted).
- **No post-race column in the features.** An assertion checks the feature list against all post-race columns: `positionOrder`, points, laps, status, finishing rank, mechanical flag, and the rest.
- **Choices on training data only.** Feature evidence, the 20% gap cutoff and every preprocessing statistic come from the training seasons.

3.3 Evidence for the features (training window 1994–2019)



- **Starting position is the strongest single signal.** `grid_pos`, `grid_pct` and `quali_position` all reach a standalone AUC of 0.905. On the same rows, grid and qualifying are almost identical (0.904 vs 0.905); both are kept because they differ exactly in the penalty cases Q1 showed matter.
- **Standings and form come close (0.85–0.88)**, and constructor versions are mostly as strong as or stronger than the driver versions. The car matters.
- `quali_gap_pct` 0.830; win shares 0.75–0.82; `experience` 0.690; `con_mech_rate_last10` 0.636.
- **Weak on their own (0.50–0.58):** `grid_penalty`, `pit_lane_start`, `home_race`, `field_size`, `driver_age`. They are tested after training rather than dropped on a hunch: in the ablation, the context group and a drop-one check for every weak feature (Section 4.6).

Figure 3.1 Standalone ROC-AUC of each feature (features where lower is better are flipped; 0.5 = no signal).

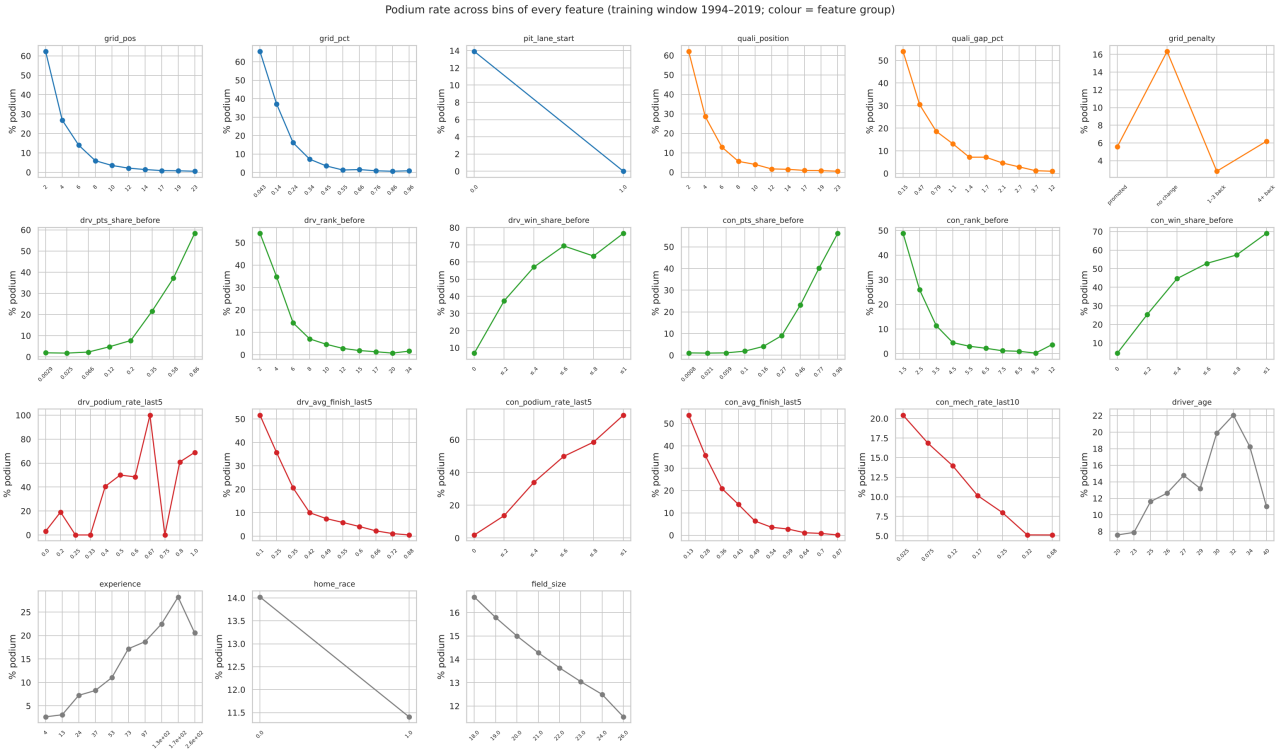


Figure 3.2 Podium rate across bins of every feature (colour = group). Most relationships are monotonic. `grid_penalty` is not meant to work alone: it matters together with qualifying position (a penalised front-row qualifier, Q1).

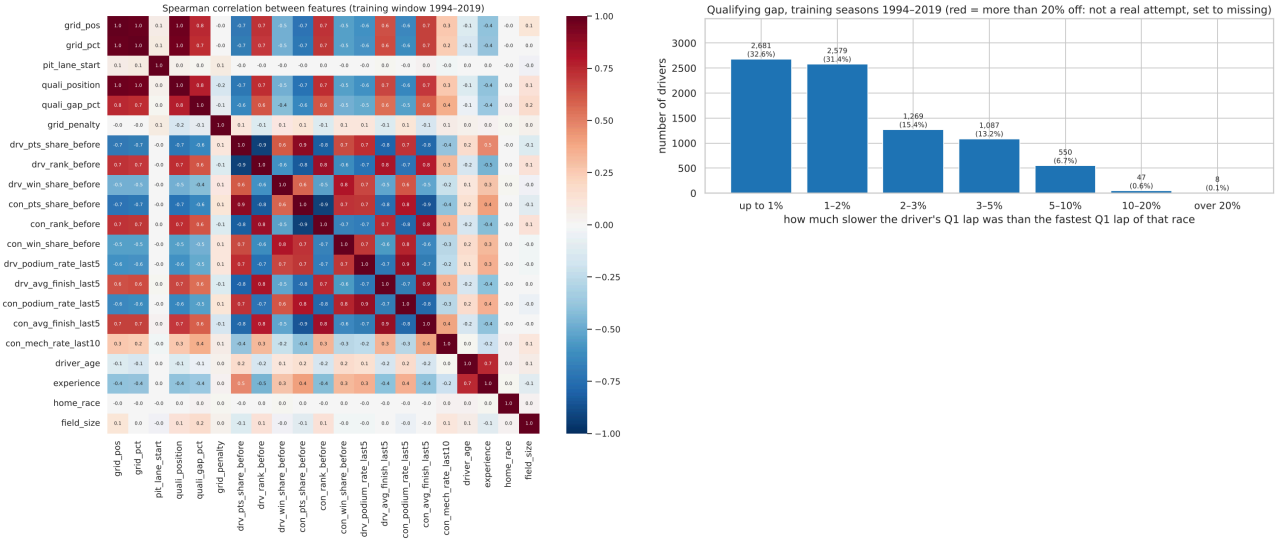


Figure 3.3 Left: Spearman correlation between features. Several features overlap strongly (grid ↔ qualifying position, rank ↔ points share, driver ↔ constructor form), so removing one group costs little, and importance is shared between correlated features. Right: the Q1 gap in the training seasons; only 8 laps are more than 20% off and are set to missing.

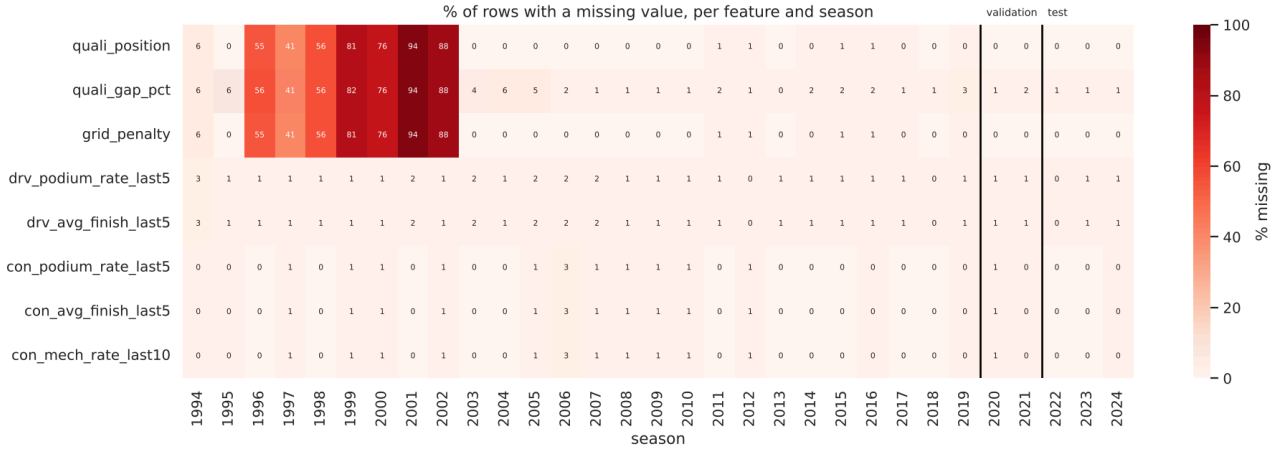


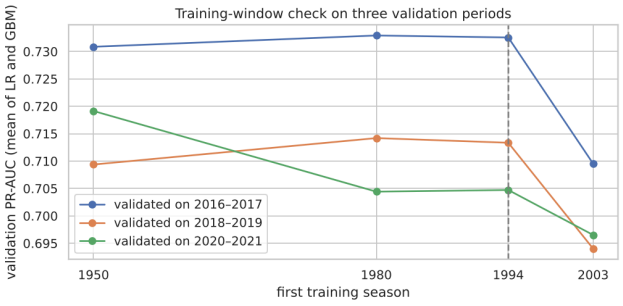
Figure 3.4 % of rows with a missing value, per feature and season. Qualifying is the only group with a large gap: 53.6% of 1994–2002 rows, against 0.3% in 2003–2019 and none in validation and test. Form is missing only at a driver's or team's first race (about 1%).

3.4 Training window and split

Split by season, not by random rows (D18). A random split would put rows of the same race into both training and test, and rows from 2023 into training while testing on 2022. That is information from the future. A season split copies real use: predict the next race from past races. It also tests the 2022 rule change (ground-effect cars).

split	seasons	races	rows	podium rate
before the window	1950–1993	537	12,420	13.0%
train	1994–2019	470	10,156	13.9%
validation	2020–2021	39	780	15.0%
test	2022–2024	68	1,359	15.0%

Table 3.2 The split and the class balance per split. An assertion checks that no race is in two splits. Seasons before 1994 are not used to fit the final models; they serve the EDA, the DE questions, the history behind `experience` and form, and the window check in Figure 3.5.



Training window 1994–2019 (D17). 1994 is the first season with qualifying data, and the sport before it differs strongly (Section 1.4). Other start years were tested on **three** validation periods (2016–17, 2018–19, 2020–21), averaging logistic regression and gradient boosting:

- no start year wins consistently;
- 1994 is within 0.001 of the best in the first two periods;
- 2003 is worst in all three, because it leaves too little data.

The window rests on the domain argument.

Figure 3.5 Validation PR-AUC by first training season, for three validation periods.

3.5 Preprocessing and its effect on the data

- **Imputation.** Missing values are filled with the **training median**, and a "was missing" indicator is added for the 8 features that have missing values, giving 29 model inputs. The medians used are: qualifying position 11, Q1 gap 1.50%, `grid_penalty` 0, driver podium rate 0, average finish 0.52, team mechanical rate 0.15.
- **Scaling.** The logistic regression and the network also get `StandardScaler`; trees do not need it.
- **Fitted on train only.** Both steps are fitted on the training rows only and then applied unchanged to validation and test (D19).

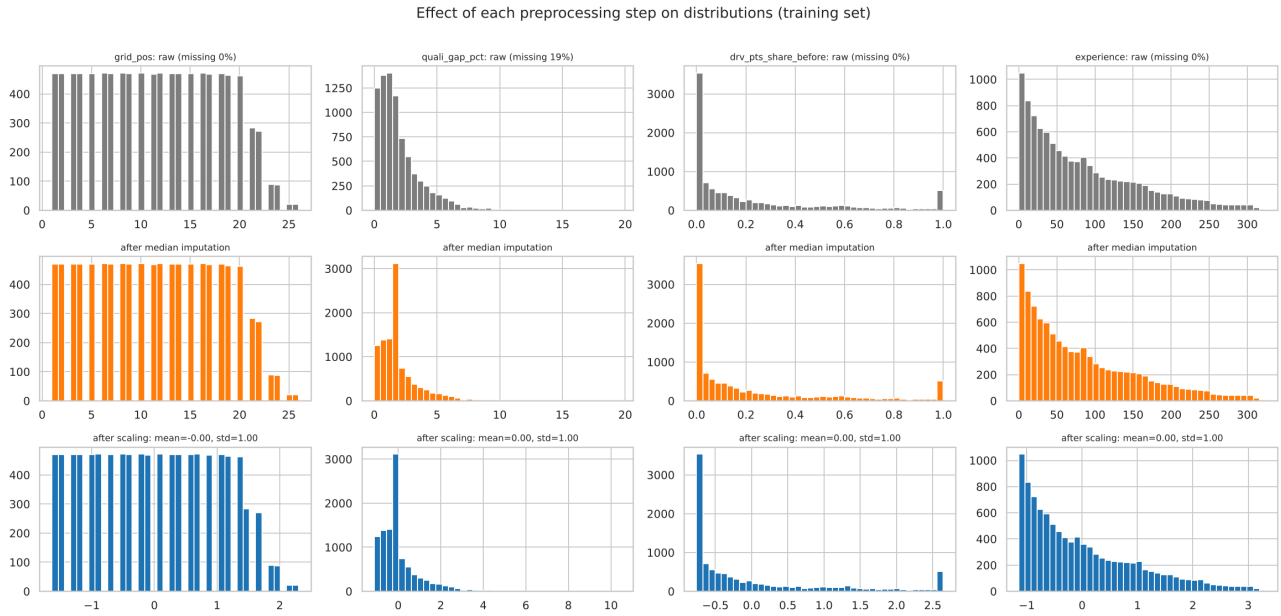


Figure 3.6 Effect of each step on four features (training set): raw, after median imputation, after scaling. Imputation creates a spike at the median for `quali_gap_pct` (19% missing); the indicator columns tell the model which values in the spike were really missing. Scaling changes position and spread, not shape.

3.6 Preprocessing-order experiments

Three experiments change only the order of two steps, with the same model, rows and threshold rule. They are decided by **correctness, not by score**: a wrong order can score as high, or higher, precisely because it uses information it should not have.

experiment	order	val PR-AUC	val ROC-AUC	val log-loss	test PR-AUC
E1 · split vs fit first (LR)	**split → fit on train (kept)**	0.7233	0.9337	0.2226	0.7089
	fit on all data → split (leaky)	0.7234	0.9338	0.2226	0.7088
E2 · impute vs scale first	**LR: impute → scale (kept)**	0.7233	0.9337	0.2226	0.7089
	LR: scale → impute	0.7230	0.9329	0.2232	0.7082
	NN: impute → scale (kept)	0.7290	0.9299	0.2268	0.7054
	NN: scale → impute	0.7204	0.9266	0.2325	0.6943
E3 · derive vs impute first (LR)	**derive → impute (kept)**	0.7233	0.9337	0.2226	0.7089
	impute → derive	0.7256	0.9311	0.2250	0.7073

Table 3.3 Preprocessing-order experiments (validation chooses; test for reference only).

- **E1, data leakage through preprocessing.** Fitting the imputer and scaler on all seasons barely changes the scores with 10,000 training rows (0.7233 vs 0.7234). But the statistics it learns come from the future: the `experience` median moves 62 → 71 and the Q1-gap median 1.50 → 1.34. The

leaky version therefore uses 2020–2024 information, and its test score cannot judge it fairly. The correct order costs nothing, so it is kept.

- **E2, order of imputation and scaling.** Imputing first squeezes the standard deviation of a feature with many missing values, so after scaling its real values are stretched further (Figure 3.7). The logistic regression is unaffected (0.7233 vs 0.7230). The network scores higher with impute → scale (0.7290 vs 0.7204), from one run per order. Impute → scale is kept: it is the standard order, and the model sees exactly mean 0 and standard deviation 1.
- **E3, derived features.** Imputing `quali_position` before computing `grid_penalty` **invents** a penalty for all 1,808 training rows without qualifying data. Its validation PR-AUC is 0.002 higher (noise), while ROC-AUC and log-loss are worse. A feature value must be real or marked as missing, so derived features are computed first and imputed afterwards.

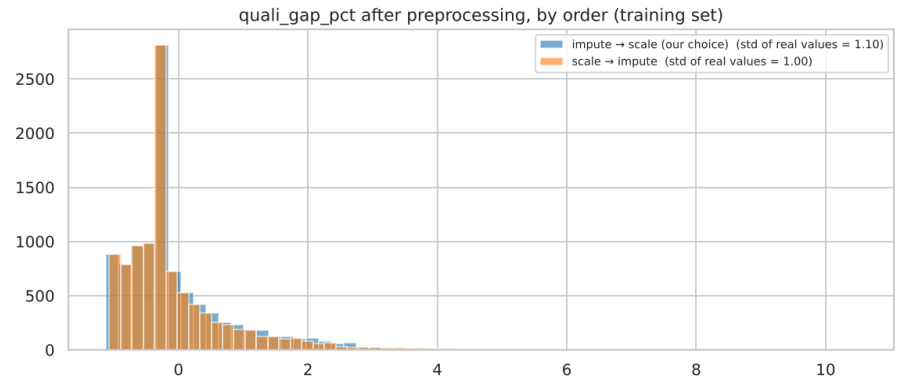


Figure 3.7 `quali_gap_pct` after preprocessing in the two E2 orders (training set).

4. Predictive modeling

Every model predicts, for one driver on the grid of one race, the probability of finishing in the top three (`positionOrder ≤ 3`). It uses only information known after qualifying and before the start.

4.1 Set-up: same rows, same features, choices on validation only

All models are trained and evaluated on the same modeling table. It has one row per driver per race, and the same 21 pre-race features in five groups: grid, qualifying, standings, form and context (Section 3). Rows are split by season (Section 3):

split	seasons	races	rows	podium rate	used for
train	1994–2019	470	10,156	13.9%	fitting the models and the preprocessing
validation	2020–2021	39	780	15.0%	thresholds, hyperparameters, early stopping, model selection
test	2022–2024	68	1,359	15.0%	scored once per model; never used to choose anything

Table 4.1 Splits. The base rate (≈ 0.15) is also the PR-AUC of a random model, so every PR-AUC below should be read against it.

Preprocessing is fitted on the training rows only and then applied unchanged to validation and test.

- **Imputation.** Missing values are filled with the training median, and a 0/1 "was missing" column is added for each of the 8 features that have missing values. The model therefore sees 29 inputs. These values are missing by design: qualifying is not recorded before 1994 and is patchy until 2002, and a driver or team has no form history at its first race.
- **Scaling.** The logistic regression and the neural network also get `StandardScaler`, because they learn by gradient steps and their penalties depend on scale. Tree models use the imputed values without scaling.

4.2 Metrics, threshold and uncertainty

- **Primary metric: PR-AUC** (average precision). Podiums are rare (≈ 15%), and PR-AUC scores how clean the *top* of the ranking is, which is the hard part of this task. A useless model still reaches ROC-AUC 0.5, but its PR-AUC is only the base rate, so PR-AUC exposes weak models. We use scikit-learn's step-wise average precision. Its documentation notes that the trapezoid version of the area "can be too optimistic".
- **Secondary metric: ROC-AUC**, for the overall separation of podium from non-podium.
- **F1, precision and recall** at one decision threshold per model. The threshold is the one that **maximises F1 on validation**; it is then used unchanged on train and test.
- **Brier score** checks calibration: whether a predicted 0.3 really means about 30%. This matters because the inference function returns a probability.
- **Uncertainty.** Validation has only 39 races, so each important number gets a **95% interval from 1,000 bootstrap resamples of races**. We resample races, not rows, because drivers in the same race are not independent. A difference between two models is resampled *paired*, with both models scored on the same redrawn races.

4.3 The models and how they work

model	how it works	settings, and how they were chosen
0. Grid rule reference, no learning	Score = 1 / grid slot: the further forward a car starts, the higher its score. It shows how much the learned models add on top of starting position.	The validation-chosen threshold is 0.333, which is exactly "podium if grid ≤ 3" (3.0 predicted podiums per race).
1. Logistic regression baseline	A weighted sum of the scaled inputs, `z = w·x + b`, turned into a probability by the sigmoid `1/(1+e^-z)`. The weights minimise log-loss plus a small L2 penalty. Each weight is the change in log-odds per standard deviation of that input, with the others held fixed.	scikit-learn default `C = 1`. That is a mild penalty for about 10,000 rows and 29 inputs, and the baseline needs no tuning.
2. Random forest	400 decision trees. Each is trained on a bootstrap sample of rows, and each split chooses from a random √29 ≈ 5 inputs. The probability is the average podium share of the leaves the row falls into. Averaging many different trees reduces variance.	Leaf size {5, 10, 20, 50, 100} × class weights {none, balanced} were tried on validation. Chosen: leaf 50, `balanced_subsample`.
3. Gradient boosting (default)	Small trees are added one after another; each new tree fits the errors (log-loss gradients) of the current model and is added with a small learning rate. Inputs are binned into histograms, which makes it fast.	scikit-learn defaults: learning rate 0.1, 100 trees, 31 leaves, minimum leaf 20.
4. Gradient boosting (tuned) hyperparameter attempt	The same algorithm.	A 54-point grid (learning rate 0.01/0.03/0.1 × trees 50/100/300 × leaves 7/15/31 × minimum leaf 20/80), each point scored on validation. Chosen: learning rate 0.01, 100 trees, 7 leaves, minimum leaf 80.
5. Shallow neural network	29 inputs → 32 ReLU → 16 ReLU → 1 sigmoid output (1,505 weights), built in Keras. Trained with Adam (learning rate 0.001, batches of 128) on binary cross-entropy. Dropout 20% and an L2 penalty of 1e-4 on the hidden layers limit overfitting.	Early stopping on validation PR-AUC (patience 15, best epoch kept). Trained with 3 seeds (42, 1, 2) to show how much the random start matters.

Table 4.2 The six modeling attempts, all on the same rows and features. Three model families plus two hyperparameter searches go beyond the brief's minimum of three distinct attempts.

Early stopping is off for gradient boosting. scikit-learn's default switches it on above 10,000 training rows (we have 10,156). It then holds out a *random* 10% of rows, which would mix drivers of the same race and make results depend on the seed. The number of trees is tuned on validation instead.

Class imbalance. The logistic regression, gradient boosting and the network are trained **without class weights**. Weights make the rare class count more, but they push every predicted probability up, and our inference function returns a probability. Studies of logistic regression report the same pattern: imbalance corrections gave poorly calibrated models that over-predicted the minority class, without improving AUROC (van den Goorbergh et al., 2022). For the random forest, weights were compared directly. The weighted forest won on validation PR-AUC by only 0.002 (0.7322 vs 0.7302), which is less than the forest's own seed spread, and its probabilities are the least calibrated of all models (validation Brier 0.106, Figure 4.4).

4.4 Results: training, validation and test

model	PR-AUC	95% CI	ROC-AUC	F1	precision	recall	Brier	threshold
0. grid rule	0.639	0.541–0.745	0.910	0.692	0.692	0.692	0.083	0.333
1. logistic regression	**0.723**	**0.613–0.837**	**0.934**	**0.708**	**0.612**	**0.838**	**0.067**	**0.218**
2. random forest	0.732	0.626–0.843	0.933	0.708	0.650	0.778	0.106	0.654
3. gradient boosting (default)	0.686	0.583–0.799	0.926	0.700	0.675	0.726	0.071	0.351
4. gradient boosting (tuned)	0.736	0.632–0.834	0.928	0.706	0.625	0.812	0.078	0.189
5. shallow NN (32–16)	0.729	0.621–0.836	0.930	0.691	0.667	0.718	0.068	0.366

Table 4.3 Validation (2020–2021), 39 races, random-model PR-AUC 0.150. Shaded row: the model used for inference (Section 4.5).

model	PR-AUC	95% CI	ROC-AUC	F1	precision	recall	Brier	podiums / race
0. grid rule	0.645	0.590–0.708	0.913	0.647	0.647	0.647	0.081	3.00
1. logistic regression	**0.709**	**0.647–0.773**	**0.936**	**0.662**	**0.531**	**0.877**	**0.069**	**4.96**
2. random forest	0.712	0.656–0.772	0.939	0.677	0.578	0.819	0.109	4.25
3. gradient boosting (default)	0.699	0.641–0.762	0.930	0.673	0.635	0.716	0.071	3.38
4. gradient boosting (tuned)	0.712	0.650–0.770	0.938	0.662	0.542	0.848	0.080	4.69
5. shallow NN (32–16)	0.705	0.647–0.763	0.935	0.656	0.599	0.725	0.070	3.63

Table 4.4 Unseen test seasons (2022–2024), 68 races, random-model PR-AUC 0.150. Each model uses its validation threshold (Table 4.3).

model	PR-AUC	ROC-AUC	F1	precision	recall	Brier
0. grid rule	0.588	0.905	0.623	0.623	0.623	0.082
1. logistic regression	**0.649**	**0.923**	**0.639**	**0.523**	**0.821**	**0.071**
2. random forest (out-of-bag)	0.648	0.924	0.647	0.551	0.785	0.109
3. gradient boosting (default)	0.940	0.989	0.844	0.775	0.926	0.034
4. gradient boosting (tuned)	0.665	0.926	0.645	0.528	0.829	0.076
5. shallow NN (32–16)	0.681	0.934	0.659	0.600	0.732	0.068

Table 4.5 Training seasons (1994–2019). The random forest's training score is its out-of-bag estimate: each row is predicted only by trees that did not see it. Its in-sample score (0.695) is inflated by memorised rows.

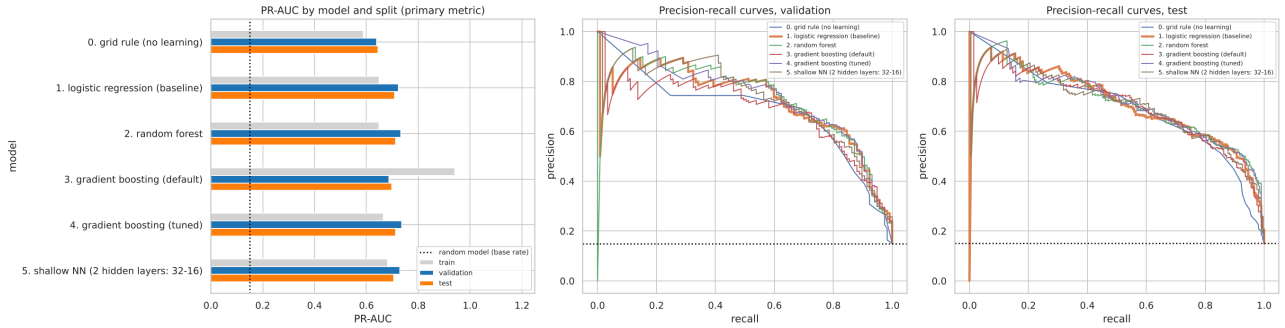


Figure 4.1 Left: PR-AUC per model on train, validation and test (dotted line = random model). Middle and right: precision-recall curves on validation and test. The thick line is the logistic regression.

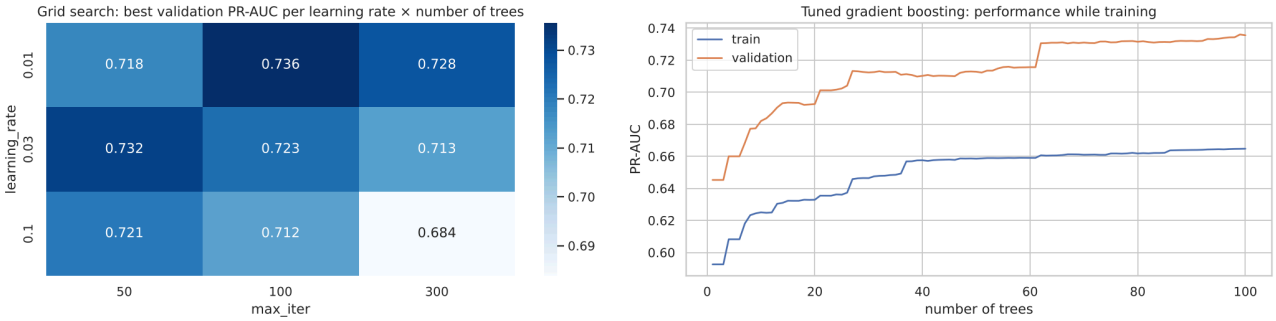


Figure 4.2 Gradient boosting. Left: best validation PR-AUC for each learning rate × number of trees in the 54-point search; the more aggressive the learning, the lower the score. Right: the tuned model's train and validation PR-AUC as trees are added; both rise together, with no sign of overfitting within 100 trees.

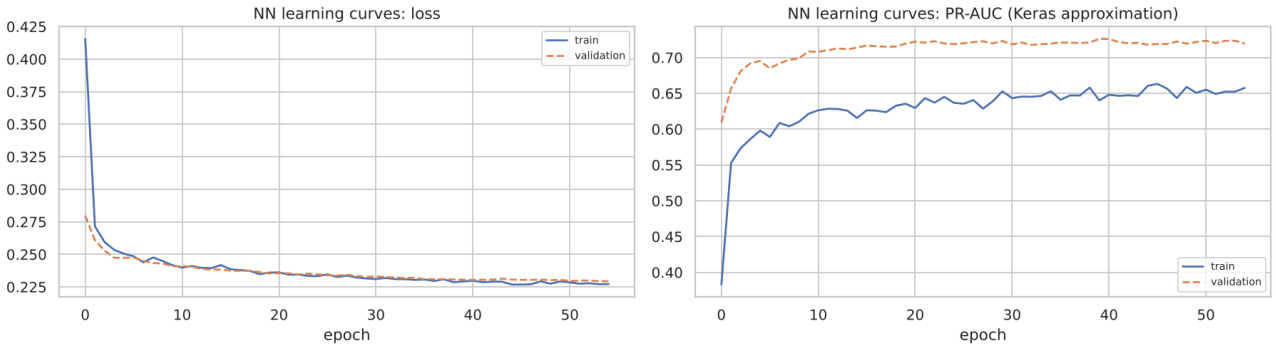


Figure 4.3 Shallow network, seed 42: loss and PR-AUC per epoch (Keras's approximate PR-AUC). Training stopped about 15 epochs after the best validation epoch, and the best epoch's weights were kept.

What training and validation show

- **Overfitting is visible, then removed.** Default gradient boosting fits the training seasons almost perfectly (train PR-AUC 0.940) but reaches only 0.686 on validation. The tuned settings learn slowly with very simple trees (0.665 train, 0.736 validation). The best point lies at the edge of the grid (slowest learning, smallest trees), which suggests that an even simpler model would do about as well.
- **The network** improves quickly in the first ~10 epochs and then flattens. Its training curve sits below validation for two reasons. First, Keras computes training metrics with dropout switched on, averaged over the epoch while the weights are still changing. Second, the training seasons are harder to rank (next point). Measured afterwards without dropout, the network's training PR-AUC is 0.681.
- **Validation is above training even for the grid rule** (0.639 vs 0.588), which learns nothing, so this is not leakage. The 1990s are much harder to rank (grid-rule PR-AUC 0.486 in 1994–99 against 0.68–0.70 in 2014–19), because fields were bigger, breakdowns more frequent and qualifying data patchier. The base rate is also lower in training (13.9% vs 15.0%).
- **Seed spread.** Over 3 seeds, the forest's validation PR-AUC ranges 0.732–0.738 (mean 0.734; test mean 0.713). The network ranges 0.726–0.729 (mean 0.727; test mean 0.701). Differences between models smaller than this spread mean nothing.

4.5 Model selection: the baseline is kept

> **Rule (validation only).** Rank the candidates by validation PR-AUC, using the mean over seeds for the forest and the network. A more complex model replaces the logistic-regression baseline **only if the paired bootstrap interval of (best – baseline) lies entirely above 0**.

- **Validation ranking.** Tuned gradient boosting 0.736, random forest 0.734, network 0.727, logistic regression 0.723, default gradient boosting 0.686.

- **Best – baseline.** +0.012, with a 95% interval of **–0.022 to +0.043**. The interval includes 0, so there is no evidence that the extra complexity helps, and the **logistic regression is the model used for inference and explanations**. This test is conservative: the tuned model is the best of 54 tries on the same 39 races, which favours it, and it still does not clear the bar.
- **Baseline – grid rule.** 95% interval **0.033 to 0.142**, entirely above 0. Standings, form and qualifying pace add real information on top of starting position.
- **Test confirms the choice was not costly.** Logistic regression 0.709, random forest 0.712, tuned gradient boosting 0.712, all well inside each other's intervals (about ±0.06). The logistic regression is also deterministic and the best calibrated (Brier 0.067 validation, 0.069 test).

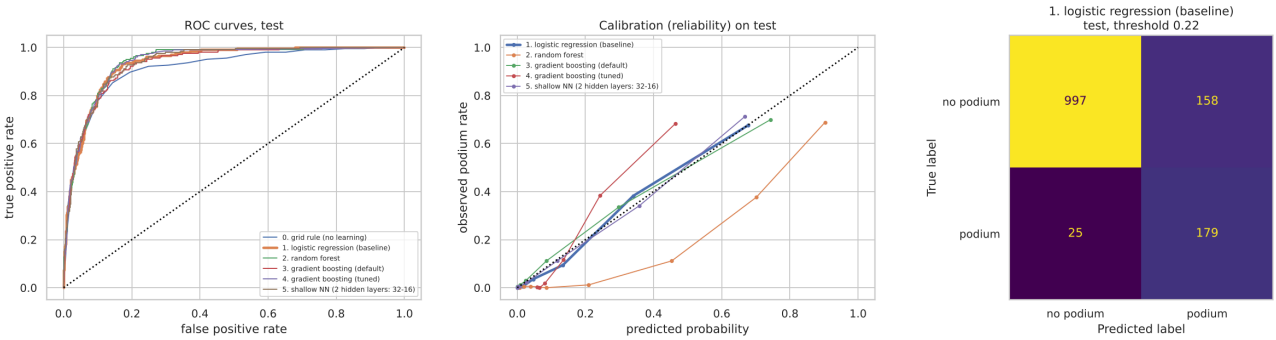


Figure 4.4 Test seasons. Left: ROC curves; the thick orange line is the logistic regression. Middle: calibration, where the points on the diagonal are well calibrated. This panel omits the grid rule, so its colours follow its own legend: the thick blue line is the logistic regression. Right: the logistic regression's confusion matrix at its validation threshold 0.22 (179 true positives, 158 false positives, 25 missed podiums).

What the comparison shows

- **The learned models clearly beat the grid rule, but not each other.** All five are inside each other's intervals on both validation and test.
- **Calibration differs a lot.** The logistic regression and the network lie close to the diagonal. The class-weighted forest predicts far above the real podium rates. Tuned gradient boosting is under-confident at the top: its highest bin predicts ≈ 0.46 where ≈ 0.68 of drivers podium. This is consistent with a slow learning rate and only 100 trees, which keep probabilities near the base rate.
- **Race top-3 hit rate.** If we take the three highest scores in each race, no learned model beats picking the first three on the grid (validation 0.692 vs 0.667–0.684; test 0.647 vs 0.637–0.647). The learned models' advantage is not in reordering the front of one grid. It is in knowing *how likely* each front-runner is to podium, for example a front-row start in the dominant car vs in a midfield car. PR-AUC and the Brier score reward exactly that.
- **Decisions at the threshold.** The baseline's F1-optimal threshold is low (0.22), so it predicts about 4.1 podiums per race on validation and 5.0 on test, more than the 3 that are possible. This follows from scoring each driver on their own (Section 4.7).

4.6 Feature-group ablation

Section 3 justified each feature *before* training. The ablation measures each group's contribution *after* training: we remove one group at a time and retrain with rows, hyperparameters and preprocessing unchanged. We run it for the inference model and for the best tree model, each change with a paired race-bootstrap interval. A group would be removed only if removing it **improved** validation PR-AUC with an interval entirely above 0.

variant	LR validation	LR test	GBM validation	GBM test
all 21 features (PR-AUC)	0.723	0.709	0.736	0.712
without grid	−0.002	**−0.010***	−0.026	**−0.026***
without qualifying	−0.008	−0.006	−0.019	+0.001
without standings	−0.011	+0.005	−0.026	+0.006
without form	−0.009	−0.000	+0.001	−0.001
without context	−0.002	+0.002	0.000	0.000
grid group only	**−0.085***	**−0.064***	**−0.098***	**−0.069***

Table 4.6 Change in PR-AUC against all features (negative = the removed group helped). * = 95% paired race-bootstrap interval excludes 0. LR = logistic regression, GBM = tuned gradient boosting.

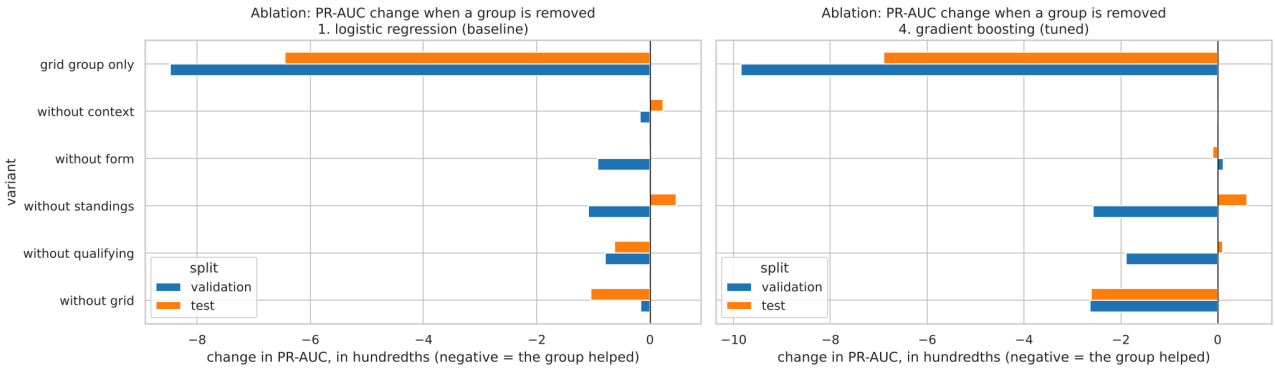


Figure 4.5 The same changes as Table 4.6, in hundredths of PR-AUC.

- **Starting position is the backbone.** Keeping only the three grid features loses 0.06–0.10 PR-AUC for both models, with intervals entirely below 0. Together, the other groups add real information on top of the grid.
- **Removing any single group changes little**, and never significantly on validation. The groups overlap (grid ↔ qualifying, standings ↔ form), so the others take over most of the removed group's job. Grid is the one group whose removal costs both models significantly on test.
- **Form and context show no measurable contribution as groups.** Gradient boosting does not use the context features at all. Removing `experience` alone does hurt the logistic regression (−0.008, interval −0.016 to −0.000). Removing the whole context group does not, because `experience` and `driver_age` carry weights of opposite sign that cancel when both are removed. No group or feature met the removal rule, so all 21 features stay.

4.7 Strengths and limitations

model	strengths	limitations
logistic regression	simple, fast, deterministic; readable weights; well-calibrated probabilities	each feature moves the log-odds along a straight line; interactions (e.g. "pole matters more at Monaco") must be built by hand
random forest	non-linear effects and interactions; robust to outliers and unscaled inputs	large; in-sample score above out-of-bag (0.695 vs 0.648); moves with the seed; class weights distort its probabilities
gradient boosting	usually strong on tabular data; handles interactions	many hyperparameters; overfits with fast learning and complex trees; tuning on 39 races is noisy; under-confident once tuned
shallow network	can approximate smooth non-linear functions	needs scaling and regularisation; moves with the seed; harder to explain; no advantage on ~10,000 tabular rows

Table 4.7 Strengths and limitations per model.

Limitations shared by all models.

- **Each driver is scored independently.** Nothing forces exactly three podiums per race, so the yes/no output predicts 4–5 per race. Racing models solve this with a race-level model such as Plackett–Luce or a conditional logit (Henderson & Kirrane, 2018), which is out of scope here.
- **No race-day information.** There is no weather, tyre strategy, safety car, crash or breakdown data, so upsets cannot be foreseen (Section 5.4).
- **Small evaluation sets.** Validation has 39 races, so model and threshold choices carry noise, and validation scores are slightly optimistic because validation was used for several choices. Test has 68 races, and its intervals are about ±0.06 wide.
- **Era shift.** The test seasons started a new set of technical rules, and the model still transfers to them. A future rule change may not behave the same way.

5. Explainability (XAI)

> **Explanations are not causal effects.** A feature being important means the *model relies on it* to predict. It does not mean the feature *causes* podiums. For example, `con_pts_share_before` is important because strong teams both score points and finish on the podium; giving a team points would not make its cars faster. Correlated features also share importance, so one feature's importance can look smaller than the information it carries. Every statement in this section describes the model's behaviour, not the sport.

5.1 What we explain, and on which data

We explain **the model used for inference**, the logistic regression (Section 4.5), because its predictions are the ones a user sees. As a cross-check we also show the global SHAP picture of tuned gradient boosting, a very different model. All explanations are computed on the **unseen test seasons (2022–2024)**. Explaining does not choose anything, so this does not leak.

- **Global:** permutation importance per feature and per feature group; SHAP summary (beeswarm) and mean |SHAP|.
- **Local:** a SHAP waterfall and LIME for the same single prediction, so the two methods can be compared, plus a SHAP waterfall for the model's most confident wrong prediction.

5.2 Permutation importance (global)

We shuffle one feature's values across the test rows, which breaks its link to the outcome, and measure how much test PR-AUC drops (20 repeats). We also shuffle **whole groups together**. This is fairer for correlated features: shuffling only `grid_pos` still leaves `grid_pct` intact.

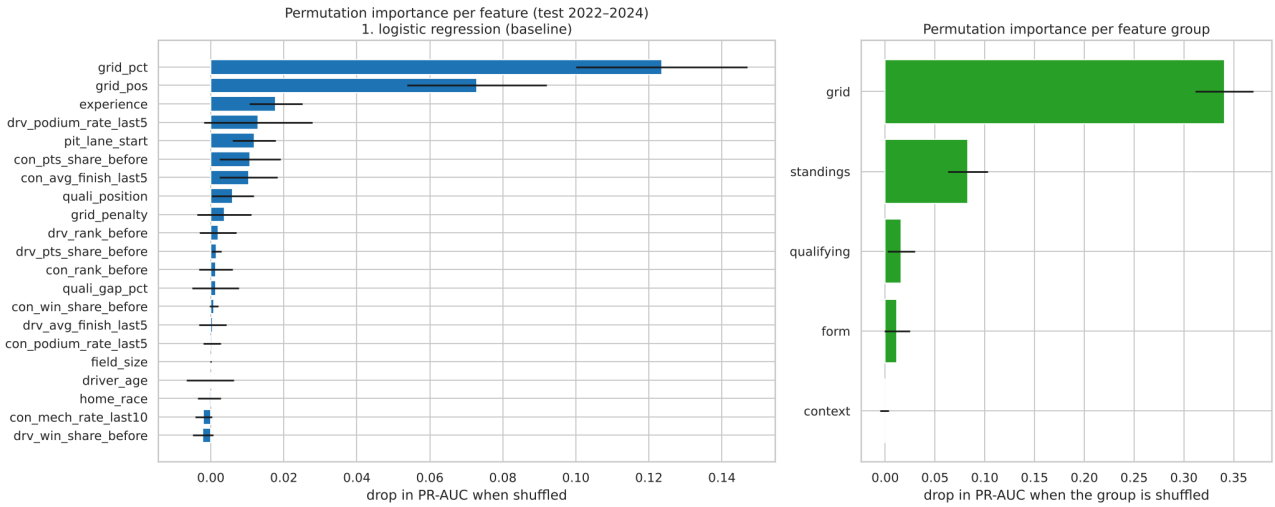


Figure 5.1 Drop in test PR-AUC when a feature (left) or a whole group (right) is shuffled; bars show the mean of 20 shuffles, lines one standard deviation.

- **Groups:** grid 0.341 (± 0.029), standings 0.083 (± 0.020), qualifying 0.016, form 0.012, context –0.001. The model depends overwhelmingly on where the car starts, then on season-long strength.
- **Single features:** only `grid_pct` (0.124), `grid_pos` (0.073), `experience` (0.018) and `pit_lane_start` (0.012) are clearly above 0. Most single features are not, even when their group matters, because a correlated partner takes over when one is shuffled. This is why the group view is the fairer one, and it agrees with the ablation.

- **`pit_lane_start` is a warning about the method.** Few test drivers started from the pit lane. Shuffling hands the flag to random drivers, including front-runners, which creates impossible rows that the model penalises strongly. Permutation importance can overstate rare, strongly weighted flags.

5.3 SHAP (global)

SHAP splits each prediction into one contribution per input. The contributions add up exactly from a **base value** (the model's output for an average training row) to the prediction. For a logistic regression, SHAP values are exact (`LinearExplainer`, 1,000 training rows as background) and are measured in **log-odds**, the scale before the sigmoid. The inputs are the 29 columns the model sees: 21 features after imputation and scaling, plus 8 missing-value indicators.

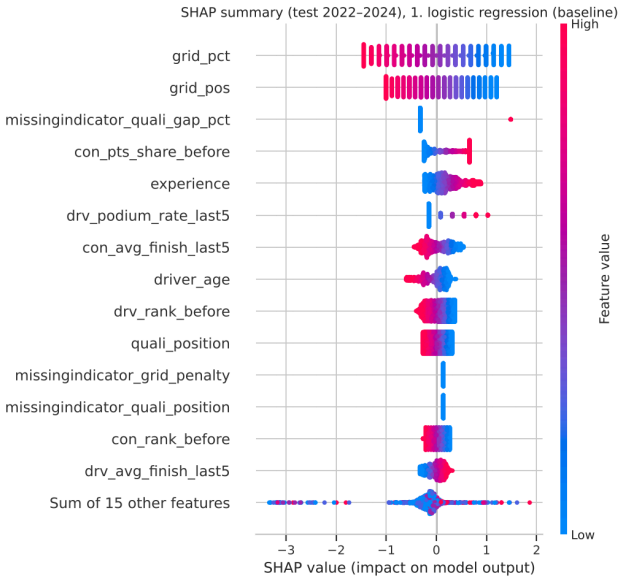


Figure 5.2 SHAP summary on test. Each dot is one driver-race. Horizontal position is how much that input pushed the prediction; colour is the input's value (red = high, blue = low).

input (scaled)	weight
grid_pct	−0.877
grid_pos	−0.738
missing: quali_gap_pct	+0.712
missing: quali_position	−0.318
missing: grid_penalty	−0.318
con_pts_share_before	+0.301
drv_podium_rate_last5	+0.291
drv_rank_before	−0.237
con_avg_finish_last5	−0.221
experience	+0.204
driver_age	−0.192

Table 5.1 The largest logistic-regression weights: change in log-odds per standard deviation, the others held fixed.

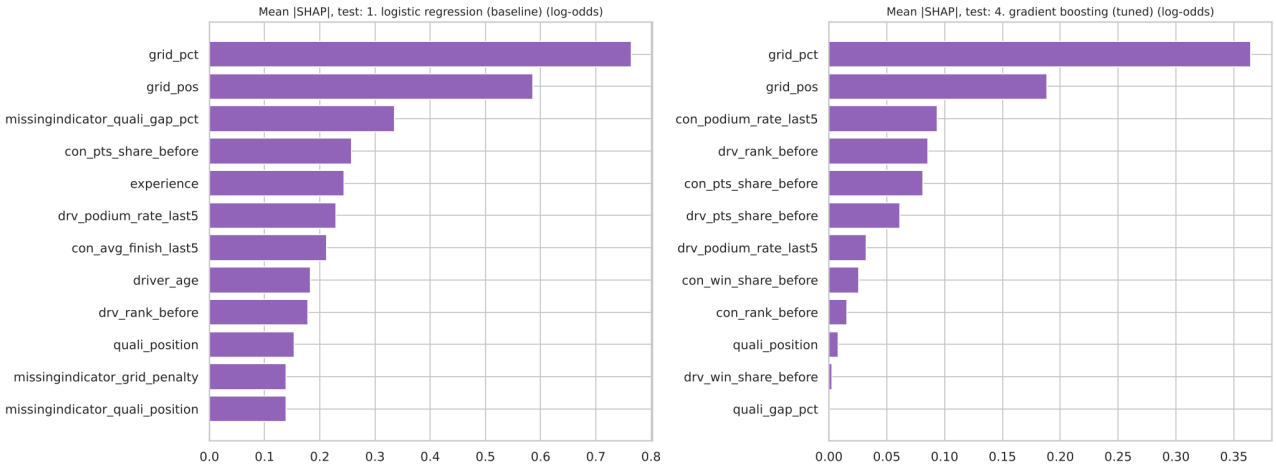


Figure 5.3 Mean |SHAP| on test for the logistic regression (left) and the tuned gradient boosting cross-check (right).

- **Starting position dominates** in both models. Gradient boosting ranks the same top two, then constructor podium rate, driver rank and constructor points share. Two very different models rely on the same information: where the car starts, and how strong it has been this season.
- **Directions are sensible.** A grid slot near the front pushes the prediction up (blue dots on the right in Figure 5.2). So do a high constructor points share, a recent podium habit and more experience. A poor constructor average finish pushes it down.

- **Some single weights point the "wrong" way.** For example, `con_mech_rate_last10`, `drv_avg_finish_last5` and `quali_gap_pct` have positive weights. These are artefacts of correlated inputs: once grid, qualifying, standings and team form are in the model, these columns only describe what is left over, and a linear model can split shared information between partners with opposite signs (e.g. age vs experience). Read the group totals, not these single numbers.
- **The large bar for the qualifying-gap missing indicator** is mostly a constant offset. Almost every test row has a qualifying time, while about 19% of training rows did not, so nearly every test row gets the same small negative value. The indicator weights are large and of opposite sign (+0.71, −0.32, −0.32) and cancel when all three are missing together, the usual pattern in the 1990s. A row missing only the Q1 time is the exception: one test row is pushed up by about +1.5 log-odds (the red dot in Figure 5.2).

5.4 Local SHAP: one prediction explained

We explain a well-known test case: **Max Verstappen at the 2023 Belgian GP**. He was fastest in qualifying, took a 5-place gearbox penalty, started 6th, and won. Model inputs (raw values): grid 6 of 20, qualifying position 1, `grid_penalty` 5; leading both championships (points share 1.00, win share 0.82, team win share 1.00); podiums in all of his last 5 races; team mechanical-failure rate 0 over its last 10 races.

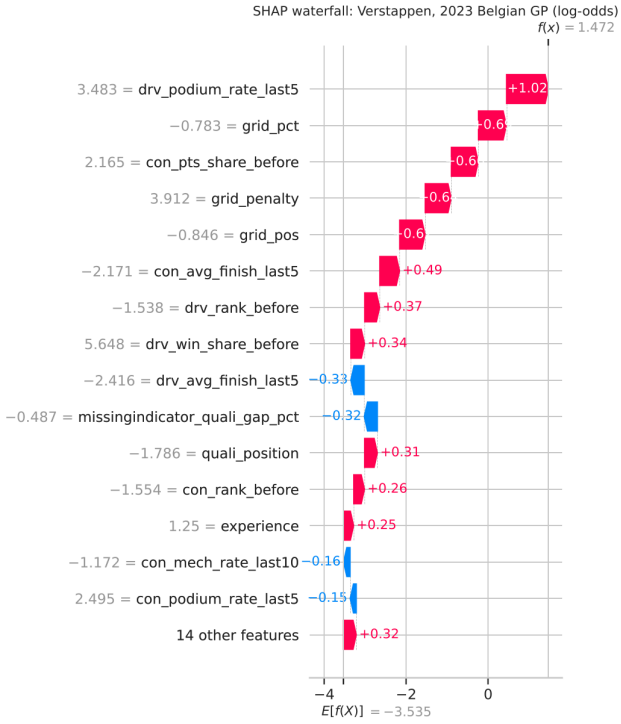


Figure 5.4 SHAP waterfall, Verstappen at the 2023 Belgian GP. The numbers left of the feature names are the *standardised* model inputs (z-scores), not raw values; the raw values are listed above.

feature group	sum of SHAP (log-odds)
standings	+1.617
grid	+1.315
form	+0.853
qualifying	+0.810
context	+0.412
all contributions	**+5.007**
base value	−3.535
prediction	**+1.472 → p = 0.813**

Table 5.2 Contributions summed per feature group.

grid slot given to the model	probability
P1 (penalty 0)	0.898
P6 (actual, penalty 5)	0.813
P10 (penalty 9)	0.713

Table 5.3 Model what-if (model behaviour, not a causal claim).

- The base value −3.535 log-odds (≈ 3% probability) is the output for an average training row. The contributions add +5.007, giving +1.472 log-odds, a **podium probability of 0.81**. He won.
- **Every group pushes up.** Standings push most (leading both championships), then grid (P6 is still far ahead of an average starter), form, qualifying and context.
- **`grid_penalty` pushes up (+0.64), not down.** For a given grid slot, a large penalty means the driver's real pace was much better than the slot suggests (he qualified first). The model has learnt that "P6 after a 5-place penalty" is a stronger position than "P6 on merit".

- The single negative bars (`drv_avg_finish_last5` and the qualifying-gap indicator) are the correlated-input and offset artefacts described in Section 5.3; their group totals are positive.

The model's most confident mistake

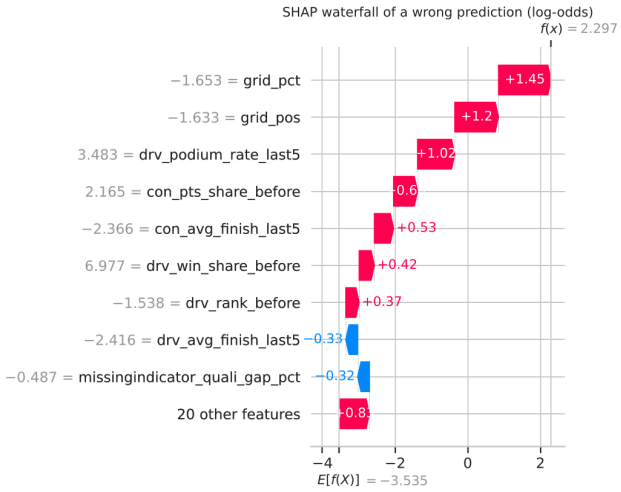


Figure 5.5 SHAP waterfall of the most confident wrong prediction (standardised inputs on the left, as in Figure 5.4).

Of all test rows that did *not* end on the podium, the highest prediction was for **Verstappen at the 2024 Australian GP**: pole position, leading both championships, probability **0.91**. He retired with a brake failure.

Every group pushed the prediction up: grid +2.66, standings +1.70, form +0.87, context +0.44 and qualifying +0.16 log-odds (–3.535 + 5.832 = 2.297 → p = 0.909).

Nothing in the pre-race data could have announced a component failure. This is the limitation of Section 4.7: the model ranks the field well, but it cannot foresee breakdowns or crashes.

5.5 LIME for the same prediction

LIME explains one prediction differently. It creates 5,000 perturbed copies of the row, asks the model about each, and fits a small weighted linear model around that point; the local model's weights are the explanation. Two properties matter when reading it:

- It describes features by **ranges** taken from the training quartiles (e.g. `grid_pos ≤ 6`), not by exact values.
- It changes features **independently**, so some synthetic rows are impossible combinations. It is also random, so we ran it with 5 seeds.

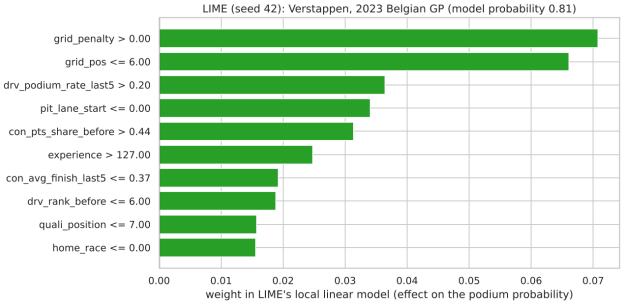


Figure 5.6 LIME (seed 42), Verstappen at the 2023 Belgian GP.

LIME condition	mean weight	seeds in top 10
grid_pos ≤ 6	0.063	5 / 5
grid_penalty > 0	0.057	5 / 5
drv_podium_rate_last5 > 0.20	0.038	5 / 5
pit_lane_start ≤ 0	0.037	5 / 5
con_pts_share_before > 0.44	0.030	5 / 5
experience > 127	0.022	5 / 5

Table 5.4 LIME weights averaged over 5 seeds. Local fit R^2 0.29–0.31; LIME's own prediction 0.25–0.28, against the model's 0.81.

- **Direction agrees with SHAP.** LIME's largest weights are the same features SHAP pointed to, with the same sign: starting near the front, a penalty that signals real pace, the recent podium rate, constructor strength and experience. They appear in the top 10 for all 5 seeds.
- **But LIME fits this point badly.** Its R^2 is about 0.3, and its own prediction (0.25–0.28) is far from the model's 0.81. LIME draws its synthetic rows from the whole training distribution, which is mostly

midfield and back-marker cars with near-zero podium chances. One straight line through them cannot follow the sigmoid's steep rise near this row, so its weights describe the model over a broad region, not at this exact point.

- **SHAP is therefore the main local explanation.** For a logistic regression it is exact and adds up to the model's output. Where both methods agree, we can be confident about what the model relied on for this prediction.

5.6 Summary of the explanations

- All methods agree that **starting position is what the model relies on most**, followed by **season-long strength** (standings) and, for single predictions, recent form. A very different model (gradient boosting) relies on the same information.
- **Group-level views are the trustworthy ones.** Correlated features share and trade importance, so single-feature numbers and single weights should not be read as separate effects.
- The local explanations show both what the model gets right (a penalised pole-sitter in the dominant car is still a likely podium) and what it cannot know (a brake failure from pole).
- Once more: these explain the **model's reasoning**, not what **causes** a podium.

6. Inference function

`predict_podium(record)` takes one driver's pre-race information as a dictionary. It returns the podium probability, a yes/no decision at the validation-tuned threshold (0.218), and the list of missing features (D27). It uses the model chosen in Section 4.5 with its own fitted preprocessing, so missing values are treated exactly as in training: the training median, plus the "was missing" indicator.

- **Required:** `grid_pos` and `field_size`. A raw grid of 0 is read as a pit-lane start (last slot plus the flag).
- **Derived when absent:** `grid_pct` from slot and field size, and `grid_penalty` from slot and qualifying position. On a 2021–22 sprint weekend, `grid_penalty` should be passed explicitly, because there the reference is the sprint result.
- **Rejected with an error:** unknown feature names (a typo would otherwise silently count as "missing"); a grid slot outside the field or not a whole number; shares and rates outside 0–1; a `grid_pct` that contradicts the slot.

demo record	probability	decision	missing features
complete: Verstappen, 2023 Belgian GP (test set)	0.813	podium (he won)	none
partially missing: the same record without the qualifying and form groups	0.840	podium	8 (3 qualifying, 5 form)
hand-written: a rookie starting 15th of 20, most history unknown	0.058	no podium	11
the same rookie from the pit lane (raw grid 0)	0.001	no podium	11

Table 6.1 The two required demonstrations (complete and partially missing record) and two hand-written records.

what is missing from the complete record	probability
nothing	0.813
qualifying group	0.677
form group	0.916
qualifying and form (the partial demo)	0.840

Table 6.2 Why the probability barely moves when half the information is gone.

- **Without qualifying**, the model no longer knows he was fastest. The position is filled with the training median (P11) plus the indicator, so the prediction falls back on grid and standings: 0.68.
- **Without form**, the probability *rises* to 0.92. In training, missing form only ever happened at a debut, so the indicator's learnt effect does not fit a driver with 174 starts.
- **With both missing**, the two effects roughly cancel (0.84).
- **The lesson:** an imputed prediction is only as good as the training data's examples of that missing pattern. That is why the function returns the list of missing features.
- **The probabilities are real probabilities:** the logistic regression lies close to the diagonal in the calibration plot (Figure 4.4).

7. Known limitations

- **Historical qualifying-coverage drift.** Qualifying data starts in 1994 and is patchy until 2002 (53.6% of 1994–2002 rows have no qualifying position), so the model learnt the qualifying features mostly from 2003 on. The missing-value indicators let it treat 1996–2002 rows differently. The earliest training

seasons are also the hardest to rank (grid-rule PR-AUC 0.49 in 1994–99, against 0.68–0.70 in 2014–2019).

- **Era sensitivity.** Rules, reliability and field sizes changed a lot. A pole start converts 51%, 65% or 81% depending on the era (Figure 1.6). The test seasons start a new set of technical rules and the model still transfers, but a future rule change may not behave the same way.
- **Small evaluation sets.** Validation has 39 races and test 68, so PR-AUC intervals are about ±0.06–0.11 wide. Differences between our learned models are inside them. Validation scores are slightly optimistic, because validation was used for several choices.
- **Independent predictions.** Each driver is scored alone. Nothing forces exactly three podiums per race, so the yes/no output predicts 4–5 per race. A race-level model (Plackett–Luce or a conditional logit) would fix this; it is out of scope for this milestone.
- **No race-day information.** Weather, tyre strategy, safety cars, crashes and breakdowns are not in the data and cause many upsets. The model's most confident miss is a pole-sitter's brake failure (Section 5.4).
- **Missing-value patterns at inference.** A record whose missing values form a pattern never seen in training (e.g. a veteran with no form data) gets an unreliable prediction (Section 6).
- **Team identity.** A renamed team gets a new `constructorId`. Its form history therefore restarts, and Q3 reports it as a separate team; a lineage view (e.g. Renault → Alpine) is a possible extension.
- **Data quirks documented but not fully fixable:**
  - Shared drives in the 1950s: the starting car cannot always be identified (D4), and one race still has four podium rows.
  - A few vacant positions after disqualifications (D8).
  - The broken race times of the 2024 Belgian GP (Section 1.3).
  - Some cars that failed before the start are coded as 0-lap retirements and count as starts (checked in Q3).
  - For raw grid 0, telling a pit-lane starter from a car that never started uses the post-race laps column (D6, D7). This affects a handful of rows.

How to reproduce

On Kaggle: attach the dataset, turn **Internet on** (the first cell installs `shap`, `lime` or `statsmodels` if missing and stops with a clear message if it cannot), and press **Run All**. The notebook finds the data under `/kaggle/input/` and writes the tables and all figures of this report to `/kaggle/working/`. The cleaning, the DE answers and the logistic regression are deterministic. The random forest and the network are seeded and report their spread over three seeds.

References

- Ergast Developer API / Kaggle: Formula 1 World Championship (1950–2024), by R. Rao.
- van den Goorbergh, R., van Smeden, M., Timmerman, D., Van Calster, B. (2022). The harm of class imbalance corrections for risk prediction models: illustration and simulation using logistic regression. *JAMIA* 29(9), 1525–1534.
- Henderson, D. A., Kirrane, L. J. (2018). A comparison of truncated and time-weighted Plackett–Luce models for probabilistic forecasting of Formula One results. *Bayesian Analysis* 13(2), 335–358.
- Lundberg, S. M., Lee, S.-I. (2017). A unified approach to interpreting model predictions. *NeurIPS*. Ribeiro, M. T., Singh, S., Guestrin, C. (2016). "Why should I trust you?" Explaining the predictions of any classifier. *KDD*.
- Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. *JASA* 22(158), 209–212.
- scikit-learn documentation (`average_precision_score`, `HistGradientBoostingClassifier`); Keras documentation (FAQ on training vs validation loss).